

Sous-suites, valeurs d'adhérence et preuve par séparation du théorème de Bolzano-Weierstrass

Charles EDOU NZE

2026-06-26

Table des matières

Exercices d'Application	5
Travaux Pratiques	27

Jalon 15 : Sous-suites, valeurs d'adhérence et preuve par séparation du théorème de Bolzano-Weierstrass

1. Genèse et Exégèse Conceptuelle

Historiquement, l'appréhension du continu et des limites a longtemps été freinée par des suites au comportement d'apparence chaotique, refusant obstinément de converger vers une limite unique. L'idée de sous-suite émerge alors comme un filtre cognitif, une extraction chirurgicale de l'ordre au sein du désordre. Si l'on imagine une foule dont la trajectoire globale semble erratique, le mathématicien, tel un observateur sélectif, choisira de ne fixer son attention que sur une sous-population spécifique. Bien que la dynamique globale fuie toute stabilisation, la trajectoire extraite révélera souvent une destination précise. C'est ici qu'intervient la puissance fondatrice du théorème de Bolzano-Weierstrass, assurant que dans un espace confiné (borné), l'infinité de l'accumulation force invariablement une convergence partielle. Cette notion est l'infrastructure même de la compacité, indispensable pour garantir l'existence de solutions à des problèmes d'optimisation.

2. Énoncé Symbolique et Typage Chirurgical

A. Anatomie des Définitions Formelles

Soit E un espace vectoriel normé (typiquement $\mathbb{R}$ ou $\mathbb{C}$) et soit $(u_n)_{n \in \mathbb{N}} \in E^{\mathbb{N}}$ une suite d'éléments de E .

1. **Extraction de Sous-suite** : Une suite $(v_k)_{k \in \mathbb{N}}$ est définie comme une *sous-suite* (ou *suite extraite*) de $(u_n)_{n \in \mathbb{N}}$ si et seulement s'il existe une application extractrice $\phi : \mathbb{N} \rightarrow \mathbb{N}$ qui est **strictement croissante** (c'est-à-dire $\forall k \in \mathbb{N}, \phi(k+1) > \phi(k)$) telle que pour tout entier naturel $k \in \mathbb{N}, v_k = u_{\phi(k)}$. L'application ϕ sélectionne les indices conservés tout en préservant leur ordre chronologique.

2. **Valeur d'Adhérence (Point d'Accumulation)** : Un scalaire $a \in E$ est qualifié de *valeur d'adhérence* de la suite $(u_n)_{n \in \mathbb{N}}$ s'il existe une extractrice ϕ telle que la sous-suite $(u_{\phi(k)})_{k \in \mathbb{N}}$ converge vers a , soit $\lim_{k \rightarrow +\infty} u_{\phi(k)} = a$.

3. **Caractérisation Topologique Stricte** : Le scalaire a est une valeur d'adhérence si et seulement si pour tout voisinage V de a , il existe une infinité d'indices n tels que $u_n \in V$. Formellement dans un espace métrique : $\forall \epsilon > 0, \forall N \in \mathbb{N}, \exists n \geq N, \|u_n - a\| < \epsilon$.

B. Exemples de Validation et Contre-exemples

- **Exemple trivial de convergence absolue** : Si (u_n) converge vers L , alors l'unique valeur d'adhérence de la suite est L , et toute sous-suite de (u_n) converge vers L . - **Exemple d'oscillation régulière** : La suite définie par $u_n = (-1)^n$ admet exactement deux valeurs d'adhérence : $\{-1, 1\}$. - **Cas pathologique d'une suite non bornée dense** : L'énumération des rationnels dans $[0, 1]$ par une suite (r_n) admet pour ensemble des valeurs d'adhérence l'intervalle $[0, 1]$ tout entier, illustrant un continuum d'adhérence. - **Cas sans adhérence** : La suite $u_n = n$ diverge vers l'infini et ne possède aucune valeur d'adhérence dans $\mathbb{R}$ (elle s'échappe de tout compact).

B. Théorèmes, Propositions & Lemmes

> **Théorème de Bolzano-Weierstrass** : > De toute suite réelle bornée, on peut extraire une sous-suite convergente. > (Plus généralement : Toute suite dans un ensemble compact admet au moins une valeur d'adhérence).

> **Lien avec la convergence** : > Une suite bornée converge si et seulement si elle admet une unique valeur d'adhérence.

3. Démonstrations

Démonstration du Théorème Pivot : Bolzano-Weierstrass par dichotomie (méthode de séparation)

Soit (u_n) une suite réelle bornée. Montrons qu'il existe une sous-suite convergente.

1. **Initialisation / Cadre :** - Puisque (u_n) est bornée, il existe un intervalle $[a_0, b_0]$ tel que $\forall n \in \mathbb{N}, u_n \in [a_0, b_0]$. - Posons $I_0 = [a_0, b_0]$ et $L_0 = b_0 - a_0$. - Nous allons construire par récurrence une suite d'intervalles emboîtés $(I_k)_{k \in \mathbb{N}}$.

2. **Étape 1 : Construction par dichotomie** Supposons $I_k = [a_k, b_k]$ construit tel qu'il contient une infinité de termes de la suite (u_n) . - Soit $m_k = \frac{a_k + b_k}{2}$ le milieu de I_k . - L'intervalle I_k est l'union de $[a_k, m_k]$ et $[m_k, b_k]$. - Puisque I_k contient une infinité de termes, l'un au moins de ces deux sous-intervalles contient aussi une infinité de termes. - On choisit $I_{k+1} = [a_{k+1}, b_{k+1}]$ comme étant ce sous-intervalle (si les deux intervalles fermés contiennent une infinité de termes, nous sélectionnons de manière déterministe le sous-intervalle de gauche $[a_k, m_k]$ afin d'assurer l'unicité de la construction de la suite d'intervalles emboîtés).

3. **Étape 2 : Propriétés des suites (a_k) et (b_k)** - Par construction, $a_k \leq a_{k+1}$ (croissante) et $b_{k+1} \leq b_k$ (décroissante). - De plus, $b_k - a_k = \frac{b_0 - a_0}{2^k} \rightarrow 0$ quand $k \rightarrow \infty$. - D'après le théorème des segments emboîtés, les suites (a_k) et (b_k) convergent vers une limite commune l .

4. **Étape 3 : Extraction de la sous-suite** - On définit $\phi(0) = 0$. - Pour chaque $k \geq 0$, on choisit $\phi(k+1)$ comme étant le plus petit entier tel que $\phi(k+1) > \phi(k)$ et $u_{\phi(k+1)} \in I_{k+1}$. Un tel entier existe car I_{k+1} contient une infinité de termes. - On a alors, pour tout k : $a_k \leq u_{\phi(k)} \leq b_k$.

5. **Conclusion :** - Par le théorème des gendarmes, comme $\lim a_k = l$ et $\lim b_k = l$, alors $\lim_{k \rightarrow \infty} u_{\phi(k)} = l$. - Nous avons extrait une sous-suite convergente. Le théorème est démontré.

4. Exercices d'Application

Exercice 1 : Application Directe (Valeurs d'adhérence)

Énoncé : Déterminer les valeurs d'adhérence de la suite $u_n = \sin(n\frac{\pi}{2})$. **Correction Détaillée :** 1. Étudions les premiers termes : - $n = 0 \implies \sin(0) = 0$ - $n = 1 \implies \sin(\pi/2) = 1$ - $n = 2 \implies \sin(\pi) = 0$ - $n = 3 \implies \sin(3\pi/2) = -1$ - $n = 4 \implies \sin(2\pi) = 0$ (Cycle de période 4). 2. Définissons des extractions : - Pour $n = 2k$: $u_{2k} = \sin(k\pi) = 0$. La sous-suite (u_{2k}) converge vers 0. - Pour $n = 4k+1$: $u_{4k+1} = \sin(2k\pi + \pi/2) = 1$. La sous-suite (u_{4k+1}) converge vers 1. - Pour $n = 4k+3$: $u_{4k+3} = \sin(2k\pi + 3\pi/2) = -1$. La sous-suite (u_{4k+3}) converge vers -1. **Conclusion :** Les valeurs d'adhérence sont $\{-1, 0, 1\}$.

Exercice 2 : Niveau Avancé (Densité)

Énoncé : Montrer que l'ensemble des valeurs d'adhérence de la suite $u_n = \cos(n)$ est l'intervalle $[-1, 1]$ tout entier. (Admettre que $\mathbb{Z} + 2\pi\mathbb{Z}$ est dense dans $\mathbb{R}$). **Correction Détaillée :** 1. Soit $l \in [-1, 1]$. Il existe $\theta \in \mathbb{R}$ tel que $\cos(\theta) = l$. 2. Par densité de $\mathbb{Z} + 2\pi\mathbb{Z}$, pour tout $\epsilon > 0$, il existe $n \in \mathbb{Z}$ et $k \in \mathbb{Z}$ tels que $|n - 2\pi k - \theta| < \epsilon$. 3. Comme la fonction $\cos$ est 1-lipschitzienne ($|\cos(x) - \cos(y)| \leq |x - y|$), on a : $|\cos(n - 2\pi k) - \cos(\theta)| \leq |n - 2\pi k - \theta| < \epsilon$. 4. Or $\cos(n - 2\pi k) = \cos(n)$. 5. Donc $|\cos(n) - l| < \epsilon$. Comme on peut trouver de tels n arbitrairement grands (par la structure de groupe dense), l est une valeur d'adhérence. **Conclusion :** Chaque point de $[-1, 1]$ est limite d'une sous-suite de $\cos(n)$.

5. Application en Intelligence Artificielle

- **Le Pont Théorique :** Le concept de compacité (Bolzano-Weierstrass) est crucial pour garantir l'**existence d'un optimum**. Si on cherche à minimiser une erreur dans un espace de paramètres borné et fermé, Bolzano-Weierstrass nous assure qu'il y a au moins un point d'accumulation où l'erreur est minimale. - **Exemple Concret :** Dans l'**Initialisation des Poids** des réseaux de neurones, on veut éviter que les signaux ne s'évaporent (Vanishing Gradient) ou n'exploient (Exploding Gradient). On s'assure que la suite des activations à travers les couches reste dans un ensemble compact. Sans cette garantie, l'algorithme d'apprentissage pourrait "diverger" vers l'infini sans jamais rencontrer de valeur d'adhérence (solution stable), rendant l'entraînement impossible.

6. Liens Sémantiques

- **Concepts Précédents requis :** Jalon 14 (Suites réelles et complexes) - **Concepts Futurs dépendants :** Jalon 35 (Caractérisation séquentielle des ouverts), Jalon 54 (Compacité générale), Jalon 129 (Optimisation)

stochastique)

Exercices d'Application

Énoncé

Soit la suite $(u_n)_{n \in \mathbb{N}}$ définie pour tout $n \in \mathbb{N}$ par :

$$u_n = \cos\left(\frac{n\pi}{3}\right)$$

1. Calculer les six premiers termes de la suite : $u_0, u_1, u_2, u_3, u_4, u_5$. 2. Montrer que la suite (u_n) est bornée. 3. Considérons la sous-suite $(v_k)_{k \in \mathbb{N}}$ définie par $v_k = u_{6k}$. Déterminer la limite de (v_k) lorsque $k \rightarrow \infty$. 4. Considérons la sous-suite $(w_k)_{k \in \mathbb{N}}$ définie par $w_k = u_{6k+2}$. Déterminer la limite de (w_k) lorsque $k \rightarrow \infty$. 5. La suite (u_n) est-elle convergente? Justifier votre réponse.

Énoncé

Soit la suite de nombres réels $(u_n)_{n \in \mathbb{N}}$ définie pour tout $n \in \mathbb{N}$ par :

$$u_n = (-1)^n + \frac{1}{n+1}$$

1. Démontrer que la suite (u_n) est bornée.
2. Identifier deux sous-suites convergentes distinctes de (u_n) et déterminer leurs limites respectives.
3. Déterminer l'ensemble de toutes les valeurs d'adhérence de la suite (u_n) .
4. Expliquer en quoi cet exercice illustre le théorème de Bolzano-Weierstrass.

—

Correction

Question 1 : Démontrer que la suite (u_n) est bornée.

Pour démontrer qu'une suite est bornée, il faut montrer qu'il existe un nombre réel $M > 0$ tel que pour tout $n \in \mathbb{N}$, $|u_n| \leq M$.

Considérons l'expression de u_n : $u_n = (-1)^n + \frac{1}{n+1}$.

Nous pouvons analyser le comportement de u_n en fonction de la parité de n .

* **Cas 1 : n est pair.** Si $n = 2k$ pour un certain $k \in \mathbb{N}$, alors $(-1)^n = (-1)^{2k} = 1$. Dans ce cas, $u_{2k} = 1 + \frac{1}{2k+1}$. Puisque $k \in \mathbb{N}$, $2k+1 \geq 1$. Donc $0 < \frac{1}{2k+1} \leq 1$. Par conséquent, $1 < u_{2k} \leq 1 + 1 = 2$.

* **Cas 2 : n est impair.** Si $n = 2k+1$ pour un certain $k \in \mathbb{N}$, alors $(-1)^n = (-1)^{2k+1} = -1$. Dans ce cas, $u_{2k+1} = -1 + \frac{1}{2k+1+1} = -1 + \frac{1}{2k+2}$. Puisque $k \in \mathbb{N}$, $2k+2 \geq 2$. Donc $0 < \frac{1}{2k+2} \leq \frac{1}{2}$. Par conséquent, $-1 < u_{2k+1} \leq -1 + \frac{1}{2} = -\frac{1}{2}$.

En combinant ces deux cas, nous observons que pour tout $n \in \mathbb{N}$: Si n est pair, $1 < u_n \leq 2$. Si n est impair, $-1 < u_n \leq -\frac{1}{2}$.

Ainsi, pour tout $n \in \mathbb{N}$, nous avons $-1 < u_n \leq 2$. Cela signifie que la suite (u_n) est bornée inférieurement par -1 et bornée supérieurement par 2 . Nous pouvons donc choisir $M = 2$ (ou n'importe quel nombre réel supérieur ou égal à 2). En effet, pour tout $n \in \mathbb{N}$, $|u_n| \leq \max(|-1|, |2|) = 2$. La suite (u_n) est donc bornée.

Question 2 : Identifier deux sous-suites convergentes distinctes de (u_n) et déterminer leurs limites respectives.

Nous allons construire deux sous-suites en sélectionnant les termes de (u_n) en fonction de la parité de leur indice.

* **Première sous-suite : $(u_{2k})_{k \in \mathbb{N}}$ (sous-suite des termes d'indices pairs).** Pour tout $k \in \mathbb{N}$, $u_{2k} = (-1)^{2k} + \frac{1}{2k+1} = 1 + \frac{1}{2k+1}$. Lorsque $k \rightarrow \infty$, le terme $\frac{1}{2k+1}$ tend vers 0. Donc, $\lim_{k \rightarrow \infty} u_{2k} = \lim_{k \rightarrow \infty} \left(1 + \frac{1}{2k+1}\right) = 1 + 0 = 1$. La sous-suite (u_{2k}) converge vers 1.

* **Deuxième sous-suite : $(u_{2k+1})_{k \in \mathbb{N}}$ (sous-suite des termes d'indices impairs).** Pour tout $k \in \mathbb{N}$, $u_{2k+1} = (-1)^{2k+1} + \frac{1}{2k+1+1} = -1 + \frac{1}{2k+2}$. Lorsque $k \rightarrow \infty$, le terme $\frac{1}{2k+2}$ tend vers 0. Donc, $\lim_{k \rightarrow \infty} u_{2k+1} = \lim_{k \rightarrow \infty} \left(-1 + \frac{1}{2k+2}\right) = -1 + 0 = -1$. La sous-suite (u_{2k+1}) converge vers -1 .

Nous avons identifié deux sous-suites convergentes distinctes : (u_{2k}) qui converge vers 1, et (u_{2k+1}) qui converge vers -1 . Leurs limites sont distinctes ($1 \neq -1$).

Question 3 : Déterminer l'ensemble de toutes les valeurs d'adhérence de la suite (u_n) .

Une valeur d'adhérence d'une suite (u_n) est la limite d'une sous-suite convergente de (u_n) . D'après la question précédente, nous savons déjà que 1 et -1 sont des valeurs d'adhérence. Il nous reste à montrer qu'il n'y en a pas d'autres.

Soit L une valeur d'adhérence de (u_n) . Par définition, il existe une sous-suite $(u_{\varphi(k)})_{k \in \mathbb{N}}$ qui converge vers L . La suite des indices $(\varphi(k))_{k \in \mathbb{N}}$ est une suite strictement croissante d'entiers naturels.

Pour chaque k , l'indice $\varphi(k)$ est soit pair, soit impair. Il y a deux possibilités pour la suite des indices $(\varphi(k))$:

1. La suite $(\varphi(k))$ contient une infinité d'indices pairs.
2. La suite $(\varphi(k))$ contient une infinité d'indices impairs.

* **Cas A : La suite $(\varphi(k))$ contient une infinité d'indices pairs.** Dans ce cas, nous pouvons extraire de $(u_{\varphi(k)})$ une sous-sous-suite $(u_{\varphi(k_j)})_{j \in \mathbb{N}}$ où tous les indices $\varphi(k_j)$ sont pairs. Pour cette sous-sous-suite, $\varphi(k_j) =$

$2m_j$ pour certains entiers m_j . Alors $u_{\varphi(k_j)} = u_{2m_j} = 1 + \frac{1}{2m_j+1}$. Puisque $(u_{\varphi(k)})$ converge vers L , toute sous-sous-suite doit également converger vers L . Donc, $\lim_{j \rightarrow \infty} u_{\varphi(k_j)} = L$. Or, $\lim_{j \rightarrow \infty} \left(1 + \frac{1}{2m_j+1}\right) = 1$. Par unicité de la limite, $L = 1$.

* **Cas B : La suite $(\varphi(k))$ contient une infinité d'indices impairs.** Dans ce cas, nous pouvons extraire de $(u_{\varphi(k)})$ une sous-sous-suite $(u_{\varphi(k_j)})_{j \in \mathbb{N}}$ où tous les indices $\varphi(k_j)$ sont impairs. Pour cette sous-sous-suite, $\varphi(k_j) = 2m_j + 1$ pour certains entiers m_j . Alors $u_{\varphi(k_j)} = u_{2m_j+1} = -1 + \frac{1}{2m_j+2}$. Puisque $(u_{\varphi(k)})$ converge vers L , toute sous-sous-suite doit également converger vers L . Donc, $\lim_{j \rightarrow \infty} u_{\varphi(k_j)} = L$. Or, $\lim_{j \rightarrow \infty} \left(-1 + \frac{1}{2m_j+2}\right) = -1$. Par unicité de la limite, $L = -1$.

Puisque la suite des indices $(\varphi(k))$ est infinie, elle doit nécessairement contenir une infinité d'indices pairs ou une infinité d'indices impairs (ou les deux). Si elle contient une infinité d'indices pairs, alors $L = 1$. Si elle contient une infinité d'indices impairs, alors $L = -1$. Il n'y a pas d'autre possibilité pour L .

Par conséquent, l'ensemble de toutes les valeurs d'adhérence de la suite (u_n) est $\{-1, 1\}$.

Question 4 : Expliquer en quoi cet exercice illustre le théorème de Bolzano-Weierstrass.

Le théorème de Bolzano-Weierstrass stipule que : **Toute suite bornée de nombres réels admet au moins une sous-suite convergente.**

Dans cet exercice, nous avons démontré les points suivants : 1. **La suite (u_n) est bornée.** (Réponse à la question 1) Nous avons montré que pour tout $n \in \mathbb{N}$, $-1 < u_n \leq 2$. La suite est donc bornée.

2. **La suite (u_n) admet au moins une sous-suite convergente.** (Réponse à la question 2) Nous avons explicitement construit deux sous-suites convergentes : * (u_{2k}) qui converge vers 1. * (u_{2k+1}) qui converge vers -1 . Le théorème de Bolzano-Weierstrass garantit l'existence d'au moins une telle sous-suite. Ici, nous en avons trouvé deux distinctes, ce qui est en accord avec le théorème.

Cet exercice illustre donc directement le théorème de Bolzano-Weierstrass en fournissant un exemple concret d'une suite bornée et en montrant, par construction, l'existence de sous-suites convergentes, comme le prédit le théorème. Il montre même qu'une suite bornée peut avoir plusieurs valeurs d'adhérence distinctes.

Exercice 3

Soit la suite réelle $(u_n)_{n \in \mathbb{N}}$ définie pour tout $n \in \mathbb{N}$ par :

$$u_n = \frac{n}{n+1} \cos\left(\frac{n\pi}{3}\right)$$

1. Montrer que la suite (u_n) est bornée. 2. En déduire, en utilisant le théorème de Bolzano-Weierstrass, l'existence d'au moins une sous-suite convergente. 3. Déterminer l'ensemble de toutes les valeurs d'adhérence de la suite (u_n) . Pour chaque valeur d'adhérence L , on précisera une sous-suite de (u_n) qui converge vers L .

Correction de l'Exercice 3

Question 1 : Montrer que la suite (u_n) est bornée.

Pour tout $n \in \mathbb{N}$, nous avons $n \geq 0$. Par conséquent, $n+1 > 0$. Le terme $\frac{n}{n+1}$ est toujours positif ou nul. De plus, $n < n+1$, donc $\frac{n}{n+1} < 1$. Ainsi, pour tout $n \in \mathbb{N}$, $0 \leq \frac{n}{n+1} < 1$.

Concernant le terme $\cos\left(\frac{n\pi}{3}\right)$, nous savons que la fonction cosinus est bornée. Pour tout $x \in \mathbb{R}$, $-1 \leq \cos(x) \leq 1$. Donc, pour tout $n \in \mathbb{N}$, $-1 \leq \cos\left(\frac{n\pi}{3}\right) \leq 1$.

En combinant ces deux inégalités, nous pouvons majorer la valeur absolue de u_n :

$$|u_n| = \left| \frac{n}{n+1} \cos\left(\frac{n\pi}{3}\right) \right| = \left| \frac{n}{n+1} \right| \cdot \left| \cos\left(\frac{n\pi}{3}\right) \right|$$

Puisque $0 \leq \frac{n}{n+1} < 1$ et $0 \leq \left| \cos\left(\frac{n\pi}{3}\right) \right| \leq 1$, nous obtenons :

$$|u_n| \leq 1 \cdot 1 = 1$$

Ainsi, pour tout $n \in \mathbb{N}$, $-1 \leq u_n \leq 1$. La suite (u_n) est donc bornée.

Question 2 : En déduire, en utilisant le théorème de Bolzano-Weierstrass, l'existence d'au moins une sous-suite convergente.

Le théorème de Bolzano-Weierstrass stipule que toute suite réelle bornée admet au moins une sous-suite convergente. D'après la question 1, nous avons montré que la suite (u_n) est bornée. Par application directe du théorème de Bolzano-Weierstrass, nous pouvons affirmer qu'il existe au moins une sous-suite de (u_n) qui converge.

Question 3 : Déterminer l'ensemble de toutes les valeurs d'adhérence de la suite (u_n) . Pour chaque valeur d'adhérence L , on précisera une sous-suite de (u_n) qui converge vers L .

Pour déterminer les valeurs d'adhérence de (u_n) , nous allons analyser le comportement du terme $\cos\left(\frac{n\pi}{3}\right)$. La fonction cosinus est 2π -périodique. Le terme $\frac{n\pi}{3}$ est 2π -périodique en n si $\frac{(n+P)\pi}{3} = \frac{n\pi}{3} + 2k\pi$ pour un entier k . Cela signifie $\frac{P\pi}{3} = 2k\pi$, soit $P = 6k$. La période minimale pour n est donc $P = 6$. Les valeurs prises par $\cos\left(\frac{n\pi}{3}\right)$ dépendent de $n \pmod{6}$: * Si $n = 6k$ (pour $k \in \mathbb{N}$), alors $\cos\left(\frac{6k\pi}{3}\right) = \cos(2k\pi) = 1$. * Si $n = 6k + 1$, alors $\cos\left(\frac{(6k+1)\pi}{3}\right) = \cos\left(2k\pi + \frac{\pi}{3}\right) = \cos\left(\frac{\pi}{3}\right) = \frac{1}{2}$. * Si $n = 6k + 2$, alors $\cos\left(\frac{(6k+2)\pi}{3}\right) = \cos\left(2k\pi + \frac{2\pi}{3}\right) = \cos\left(\frac{2\pi}{3}\right) = -\frac{1}{2}$. * Si $n = 6k + 3$, alors $\cos\left(\frac{(6k+3)\pi}{3}\right) = \cos(2k\pi + \pi) = \cos(\pi) = -1$. * Si $n = 6k + 4$, alors $\cos\left(\frac{(6k+4)\pi}{3}\right) = \cos\left(2k\pi + \frac{4\pi}{3}\right) = \cos\left(\frac{4\pi}{3}\right) = -\frac{1}{2}$. * Si $n = 6k + 5$, alors $\cos\left(\frac{(6k+5)\pi}{3}\right) = \cos\left(2k\pi + \frac{5\pi}{3}\right) = \cos\left(\frac{5\pi}{3}\right) = \frac{1}{2}$.

Les valeurs prises par $\cos\left(\frac{n\pi}{3}\right)$ sont donc cycliquement $1, \frac{1}{2}, -\frac{1}{2}, -1, -\frac{1}{2}, \frac{1}{2}, \dots$. L'ensemble des valeurs prises par $\cos\left(\frac{n\pi}{3}\right)$ est fini : $\left\{1, \frac{1}{2}, -\frac{1}{2}, -1\right\}$.

Considérons maintenant le comportement du terme $\frac{n}{n+1}$ lorsque $n \rightarrow \infty$. Nous avons $\lim_{n \rightarrow \infty} \frac{n}{n+1} = \lim_{n \rightarrow \infty} \frac{1}{1+\frac{1}{n}} = 1$.

Nous allons construire des sous-suites de (u_n) en sélectionnant les indices n selon leur reste modulo 6.

1. **Sous-suite pour $n = 6k$:** Soit la sous-suite $(u_{6k})_{k \in \mathbb{N}}$.

$$u_{6k} = \frac{6k}{6k+1} \cos\left(\frac{6k\pi}{3}\right) = \frac{6k}{6k+1} \cdot 1$$

Lorsque $k \rightarrow \infty$, $\frac{6k}{6k+1} \rightarrow 1$. Donc, $\lim_{k \rightarrow \infty} u_{6k} = 1 \cdot 1 = 1$. Ainsi, $L_1 = 1$ est une valeur d'adhérence de (u_n) .

2. Sous-suite pour $n = 6k + 1$: Soit la sous-suite $(u_{6k+1})_{k \in \mathbb{N}}$.

$$u_{6k+1} = \frac{6k+1}{6k+1+1} \cos\left(\frac{(6k+1)\pi}{3}\right) = \frac{6k+1}{6k+2} \cdot \frac{1}{2}$$

Lorsque $k \rightarrow \infty$, $\frac{6k+1}{6k+2} = \frac{6+1/k}{6+2/k} \rightarrow \frac{6}{6} = 1$. Donc, $\lim_{k \rightarrow \infty} u_{6k+1} = 1 \cdot \frac{1}{2} = \frac{1}{2}$. Ainsi, $L_2 = \frac{1}{2}$ est une valeur d'adhérence de (u_n) .

3. Sous-suite pour $n = 6k + 2$: Soit la sous-suite $(u_{6k+2})_{k \in \mathbb{N}}$.

$$u_{6k+2} = \frac{6k+2}{6k+2+1} \cos\left(\frac{(6k+2)\pi}{3}\right) = \frac{6k+2}{6k+3} \cdot \left(-\frac{1}{2}\right)$$

Lorsque $k \rightarrow \infty$, $\frac{6k+2}{6k+3} = \frac{6+2/k}{6+3/k} \rightarrow \frac{6}{6} = 1$. Donc, $\lim_{k \rightarrow \infty} u_{6k+2} = 1 \cdot \left(-\frac{1}{2}\right) = -\frac{1}{2}$. Ainsi, $L_3 = -\frac{1}{2}$ est une valeur d'adhérence de (u_n) .

4. Sous-suite pour $n = 6k + 3$: Soit la sous-suite $(u_{6k+3})_{k \in \mathbb{N}}$.

$$u_{6k+3} = \frac{6k+3}{6k+3+1} \cos\left(\frac{(6k+3)\pi}{3}\right) = \frac{6k+3}{6k+4} \cdot (-1)$$

Lorsque $k \rightarrow \infty$, $\frac{6k+3}{6k+4} = \frac{6+3/k}{6+4/k} \rightarrow \frac{6}{6} = 1$. Donc, $\lim_{k \rightarrow \infty} u_{6k+3} = 1 \cdot (-1) = -1$. Ainsi, $L_4 = -1$ est une valeur d'adhérence de (u_n) .

5. Sous-suite pour $n = 6k + 4$: Soit la sous-suite $(u_{6k+4})_{k \in \mathbb{N}}$.

$$u_{6k+4} = \frac{6k+4}{6k+4+1} \cos\left(\frac{(6k+4)\pi}{3}\right) = \frac{6k+4}{6k+5} \cdot \left(-\frac{1}{2}\right)$$

Lorsque $k \rightarrow \infty$, $\frac{6k+4}{6k+5} = \frac{6+4/k}{6+5/k} \rightarrow \frac{6}{6} = 1$. Donc, $\lim_{k \rightarrow \infty} u_{6k+4} = 1 \cdot \left(-\frac{1}{2}\right) = -\frac{1}{2}$. Cette valeur d'adhérence est déjà trouvée (L_3).

6. Sous-suite pour $n = 6k + 5$: Soit la sous-suite $(u_{6k+5})_{k \in \mathbb{N}}$.

$$u_{6k+5} = \frac{6k+5}{6k+5+1} \cos\left(\frac{(6k+5)\pi}{3}\right) = \frac{6k+5}{6k+6} \cdot \frac{1}{2}$$

Lorsque $k \rightarrow \infty$, $\frac{6k+5}{6k+6} = \frac{6+5/k}{6+6/k} \rightarrow \frac{6}{6} = 1$. Donc, $\lim_{k \rightarrow \infty} u_{6k+5} = 1 \cdot \frac{1}{2} = \frac{1}{2}$. Cette valeur d'adhérence est déjà trouvée (L_2).

L'ensemble des valeurs d'adhérence que nous avons identifiées est $\{1, \frac{1}{2}, -\frac{1}{2}, -1\}$.

Pour montrer que ce sont *toutes* les valeurs d'adhérence, considérons une sous-suite quelconque $(u_{\phi(k)})$ de (u_n) qui converge vers une limite L . Nous avons $u_{\phi(k)} = \frac{\phi(k)}{\phi(k)+1} \cos\left(\frac{\phi(k)\pi}{3}\right)$. Puisque $\lim_{k \rightarrow \infty} \frac{\phi(k)}{\phi(k)+1} = 1$ (car $\phi(k) \rightarrow \infty$ lorsque $k \rightarrow \infty$), et que $\lim_{k \rightarrow \infty} u_{\phi(k)} = L$, il s'ensuit que la sous-suite $\left(\cos\left(\frac{\phi(k)\pi}{3}\right)\right)_{k \in \mathbb{N}}$ doit également converger vers L . Or, la suite $\left(\cos\left(\frac{n\pi}{3}\right)\right)_{n \in \mathbb{N}}$ ne prend qu'un nombre fini de valeurs : $\{1, \frac{1}{2}, -\frac{1}{2}, -1\}$. Si une sous-suite d'une suite ne prenant qu'un nombre fini de valeurs converge, alors sa limite doit nécessairement être l'une de ces valeurs. Par conséquent, L doit appartenir à l'ensemble $\{1, \frac{1}{2}, -\frac{1}{2}, -1\}$.

L'ensemble de toutes les valeurs d'adhérence de la suite (u_n) est donc exactement $\{1, \frac{1}{2}, -\frac{1}{2}, -1\}$.

Énoncé

Soit la suite $(u_n)_{n \in \mathbb{N}^*}$ définie pour tout $n \in \mathbb{N}^*$ par :

$$u_n = \left(1 + \frac{(-1)^n}{n}\right) \sin\left(\frac{n\pi}{2}\right)$$

1. Démontrer que la suite (u_n) admet au moins une sous-suite convergente. 2. Déterminer l'ensemble de toutes les valeurs d'adhérence de la suite (u_n) . 3. En déduire la limite supérieure ($\limsup u_n$) et la limite inférieure ($\liminf u_n$) de la suite (u_n) .

Énoncé

Soit la suite $(u_n)_{n \in \mathbb{N}}$ définie pour tout $n \in \mathbb{N}$ par :

$$u_n = \sin\left(\frac{n\pi}{3}\right) + \cos\left(\frac{n\pi}{2}\right)$$

1. Montrer que la suite $(u_n)_{n \in \mathbb{N}}$ est bornée. 2. Déterminer l'ensemble A des valeurs d'adhérence de la suite $(u_n)_{n \in \mathbb{N}}$. 3. Déterminer $\limsup_{n \rightarrow \infty} u_n$ et $\liminf_{n \rightarrow \infty} u_n$. 4. La suite $(u_n)_{n \in \mathbb{N}}$ converge-t-elle? Justifier votre réponse.

Exercice 6

Soit la suite réelle $(x_n)_{n \in \mathbb{N}}$ définie pour tout $n \in \mathbb{N}$ par :

$$x_n = \frac{n \pmod{3}}{n+1} + \cos\left(\frac{n\pi}{2}\right)$$

où $n \pmod{3}$ désigne le reste de la division euclidienne de n par 3.

1. Démontrer que la suite (x_n) est bornée. 2. En déduire, en utilisant le théorème de Bolzano-Weierstrass, que la suite (x_n) admet au moins une sous-suite convergente. 3. Déterminer l'ensemble de toutes les valeurs d'adhérence de la suite (x_n) . 4. En déduire la limite supérieure et la limite inférieure de la suite (x_n) .

Correction de l'Exercice 6

Question 1 : Démontrer que la suite (x_n) est bornée.

Pour tout $n \in \mathbb{N}$, la suite (x_n) est définie par $x_n = \frac{n \pmod{3}}{n+1} + \cos\left(\frac{n\pi}{2}\right)$.

Nous allons analyser les bornes de chacun des deux termes composant x_n .

* **Analyse du terme $\frac{n \pmod{3}}{n+1}$** : Le reste de la division euclidienne de n par 3, noté $n \pmod{3}$, est un entier qui appartient à l'ensemble $\{0, 1, 2\}$. Ainsi, pour tout $n \in \mathbb{N}$, nous avons l'encadrement suivant :

$$0 \leq n \pmod{3} \leq 2$$

Le dénominateur $n+1$ est toujours strictement positif pour $n \in \mathbb{N}$ (il vaut au minimum 1 pour $n=0$). Nous pouvons donc diviser l'inégalité par $n+1$ sans en changer le sens :

$$\frac{0}{n+1} \leq \frac{n \pmod{3}}{n+1} \leq \frac{2}{n+1}$$

$$0 \leq \frac{n \pmod{3}}{n+1} \leq \frac{2}{n+1}$$

Pour $n=0$, le terme vaut $\frac{0}{1} = 0$. Pour $n \geq 0$, $n+1 \geq 1$, ce qui implique $\frac{1}{n+1} \leq 1$. Par conséquent, $\frac{2}{n+1} \leq 2$. Donc, pour tout $n \in \mathbb{N}$, nous avons :

$$0 \leq \frac{n \pmod{3}}{n+1} \leq 2$$

* **Analyse du terme $\cos\left(\frac{n\pi}{2}\right)$** : La fonction cosinus est une fonction bornée. Pour tout argument réel X , nous savons que $-1 \leq \cos(X) \leq 1$. En particulier, pour $X = \frac{n\pi}{2}$, nous avons :

$$-1 \leq \cos\left(\frac{n\pi}{2}\right) \leq 1$$

* **Combinaison des bornes** : En additionnant les inégalités obtenues pour les deux termes, nous obtenons un encadrement pour x_n :

$$0 + (-1) \leq \frac{n \pmod{3}}{n+1} + \cos\left(\frac{n\pi}{2}\right) \leq 2 + 1$$
$$-1 \leq x_n \leq 3$$

Puisque tous les termes de la suite (x_n) sont compris entre -1 et 3 , la suite (x_n) est bornée. Par exemple, on peut affirmer que $|x_n| \leq 3$ pour tout $n \in \mathbb{N}$.

Question 2 : En déduire, en utilisant le théorème de Bolzano-Weierstrass, que la suite (x_n) admet au moins une sous-suite convergente.

Le théorème de Bolzano-Weierstrass est un résultat fondamental de l'analyse réelle. Il stipule que toute suite réelle bornée admet au moins une sous-suite convergente.

D'après la question 1, nous avons démontré que la suite (x_n) est une suite de nombres réels et qu'elle est bornée. Par conséquent, en appliquant directement le théorème de Bolzano-Weierstrass, nous pouvons conclure que la suite (x_n) admet au moins une sous-suite convergente.

Question 3 : Déterminer l'ensemble de toutes les valeurs d'adhérence de la suite (x_n) .

Soit $y_n = \frac{n \pmod{3}}{n+1}$ et $z_n = \cos\left(\frac{n\pi}{2}\right)$. Ainsi, $x_n = y_n + z_n$.

Nous allons d'abord déterminer la limite de y_n lorsque $n \rightarrow \infty$. D'après la question 1, nous avons l'encadrement $0 \leq y_n \leq \frac{2}{n+1}$. Puisque $\lim_{n \rightarrow \infty} 0 = 0$ et $\lim_{n \rightarrow \infty} \frac{2}{n+1} = 0$, le théorème des gendarmes (ou théorème d'encadrement) nous permet de conclure que :

$$\lim_{n \rightarrow \infty} y_n = 0$$

Ensuite, nous allons analyser le comportement de $z_n = \cos\left(\frac{n\pi}{2}\right)$. Les valeurs de z_n dépendent du reste de la division de n par 4. Nous allons examiner les quatre cas possibles pour $n \pmod{4}$:

* Si $n = 4k$ pour un entier $k \geq 0$: $z_{4k} = \cos\left(\frac{4k\pi}{2}\right) = \cos(2k\pi) = 1$. * Si $n = 4k + 1$ pour un entier $k \geq 0$: $z_{4k+1} = \cos\left(\frac{(4k+1)\pi}{2}\right) = \cos\left(2k\pi + \frac{\pi}{2}\right) = \cos\left(\frac{\pi}{2}\right) = 0$. * Si $n = 4k + 2$ pour un entier $k \geq 0$: $z_{4k+2} = \cos\left(\frac{(4k+2)\pi}{2}\right) = \cos(2k\pi + \pi) = \cos(\pi) = -1$. * Si $n = 4k + 3$ pour un entier $k \geq 0$: $z_{4k+3} = \cos\left(\frac{(4k+3)\pi}{2}\right) = \cos\left(2k\pi + \frac{3\pi}{2}\right) = \cos\left(\frac{3\pi}{2}\right) = 0$.

La suite (z_n) prend donc les valeurs $1, 0, -1, 0, 1, 0, -1, 0, \dots$ de manière cyclique. L'ensemble des valeurs d'adhérence de la suite (z_n) est $\{-1, 0, 1\}$.

Maintenant, nous allons déterminer les valeurs d'adhérence de (x_n) en considérant des sous-suites basées sur $n \pmod{4}$. Pour chaque cas, nous utiliserons le fait que $\lim_{n \rightarrow \infty} y_n = 0$, ce qui implique que $\lim_{k \rightarrow \infty} y_{\phi(k)} = 0$ pour toute sous-suite $(y_{\phi(k)})$.

1. **Considérons la sous-suite (x_{4k}) pour $k \in \mathbb{N}$:** $x_{4k} = \frac{4k \pmod{3}}{4k+1} + \cos\left(\frac{4k\pi}{2}\right)$. Puisque $4k = 3k + k$, on a $4k \pmod{3} = k \pmod{3}$. Donc, $x_{4k} = \frac{k \pmod{3}}{4k+1} + 1$. Nous savons que $0 \leq k \pmod{3} \leq 2$. Par conséquent, $0 \leq \frac{k \pmod{3}}{4k+1} \leq \frac{2}{4k+1}$. Comme $\lim_{k \rightarrow \infty} \frac{2}{4k+1} = 0$, par le théorème des gendarmes, $\lim_{k \rightarrow \infty} \frac{k \pmod{3}}{4k+1} = 0$. Ainsi, $\lim_{k \rightarrow \infty} x_{4k} = 0 + 1 = 1$. La valeur 1 est donc une valeur d'adhérence de la suite (x_n) .

2. **Considérons la sous-suite (x_{4k+1}) pour $k \in \mathbb{N}$:** $x_{4k+1} = \frac{(4k+1) \pmod{3}}{(4k+1)+1} + \cos\left(\frac{(4k+1)\pi}{2}\right)$. Puisque $4k+1 = 3k + k + 1$, on a $(4k+1) \pmod{3} = (k+1) \pmod{3}$. Donc, $x_{4k+1} = \frac{(k+1) \pmod{3}}{4k+2} + 0$. Nous savons que $0 \leq (k+1) \pmod{3} \leq 2$. Par conséquent, $0 \leq \frac{(k+1) \pmod{3}}{4k+2} \leq \frac{2}{4k+2}$. Comme $\lim_{k \rightarrow \infty} \frac{2}{4k+2} = 0$, par le théorème des gendarmes, $\lim_{k \rightarrow \infty} \frac{(k+1) \pmod{3}}{4k+2} = 0$. Ainsi, $\lim_{k \rightarrow \infty} x_{4k+1} = 0 + 0 = 0$. La valeur 0 est donc une valeur d'adhérence de la suite (x_n) .

3. **Considérons la sous-suite (x_{4k+2}) pour $k \in \mathbb{N}$:** $x_{4k+2} = \frac{(4k+2) \pmod{3}}{(4k+2)+1} + \cos\left(\frac{(4k+2)\pi}{2}\right)$. Puisque $4k+2 = 3k + k + 2$, on a $(4k+2) \pmod{3} = (k+2) \pmod{3}$. Donc, $x_{4k+2} = \frac{(k+2) \pmod{3}}{4k+3} + (-1)$. Nous savons que $0 \leq (k+2) \pmod{3} \leq 2$. Par conséquent, $0 \leq \frac{(k+2) \pmod{3}}{4k+3} \leq \frac{2}{4k+3}$. Comme $\lim_{k \rightarrow \infty} \frac{2}{4k+3} = 0$, par le théorème des gendarmes, $\lim_{k \rightarrow \infty} \frac{(k+2) \pmod{3}}{4k+3} = 0$. Ainsi, $\lim_{k \rightarrow \infty} x_{4k+2} = 0 + (-1) = -1$. La valeur -1 est donc une valeur d'adhérence de la suite (x_n) .

4. **Considérons la sous-suite (x_{4k+3}) pour $k \in \mathbb{N}$:** $x_{4k+3} = \frac{(4k+3) \pmod{3}}{(4k+3)+1} + \cos\left(\frac{(4k+3)\pi}{2}\right)$. Puisque $4k+3 = 3k + k + 3$, on a $(4k+3) \pmod{3} = k \pmod{3}$. Donc, $x_{4k+3} = \frac{k \pmod{3}}{4k+4} + 0$. Nous savons que $0 \leq k \pmod{3} \leq 2$. Par conséquent, $0 \leq \frac{k \pmod{3}}{4k+4} \leq \frac{2}{4k+4}$. Comme $\lim_{k \rightarrow \infty} \frac{2}{4k+4} = 0$, par le théorème des gendarmes, $\lim_{k \rightarrow \infty} \frac{k \pmod{3}}{4k+4} = 0$. Ainsi, $\lim_{k \rightarrow \infty} x_{4k+3} = 0 + 0 = 0$. Cette sous-suite converge vers 0, ce qui confirme que 0 est une valeur d'adhérence.

Nous avons identifié trois valeurs d'adhérence pour la suite (x_n) : $\{-1, 0, 1\}$. Pour montrer que ce sont les *seules* valeurs d'adhérence, considérons une sous-suite quelconque $(x_{\phi(k)})$ qui converge vers une limite L . Nous avons $x_{\phi(k)} = y_{\phi(k)} + z_{\phi(k)}$. Puisque $\lim_{n \rightarrow \infty} y_n = 0$, toute sous-suite $(y_{\phi(k)})$ converge également vers 0. Par conséquent, la limite L de $(x_{\phi(k)})$ est donnée par :

$$L = \lim_{k \rightarrow \infty} x_{\phi(k)} = \lim_{k \rightarrow \infty} (y_{\phi(k)} + z_{\phi(k)}) = \lim_{k \rightarrow \infty} y_{\phi(k)} + \lim_{k \rightarrow \infty} z_{\phi(k)} = 0 + \lim_{k \rightarrow \infty} z_{\phi(k)}$$

Donc, $L = \lim_{k \rightarrow \infty} z_{\phi(k)}$. La suite (z_n) ne prend que les valeurs $1, 0, -1, 0, \dots$. Toute sous-suite convergente de (z_n) doit nécessairement converger vers l'une de ces valeurs qui apparaissent une infinité de fois. C'est-à-dire, la limite d'une sous-suite convergente de (z_n) doit être une valeur d'adhérence de (z_n) . L'ensemble des valeurs d'adhérence de (z_n) est précisément $\{-1, 0, 1\}$. Par conséquent, la limite L de toute sous-suite convergente de (x_n) doit appartenir à l'ensemble $\{-1, 0, 1\}$.

L'ensemble de toutes les valeurs d'adhérence de la suite (x_n) est donc $\{-1, 0, 1\}$.

Question 4 : En déduire la limite supérieure et la limite inférieure de la suite (x_n) .

Pour une suite réelle bornée, la limite supérieure est la plus grande de ses valeurs d'adhérence, et la limite inférieure est la plus petite de ses valeurs d'adhérence.

D'après la question 3, l'ensemble des valeurs d'adhérence de la suite (x_n) est $\{-1, 0, 1\}$.

* La limite supérieure de (x_n) , notée $\limsup_{n \rightarrow \infty} x_n$, est la plus grande valeur dans l'ensemble des valeurs d'adhérence :

$$\limsup_{n \rightarrow \infty} x_n = \max(\{-1, 0, 1\}) = 1$$

* La limite inférieure de (x_n) , notée $\liminf_{n \rightarrow \infty} x_n$, est la plus petite valeur dans l'ensemble des valeurs d'adhérence :

$$\liminf_{n \rightarrow \infty} x_n = \min(\{-1, 0, 1\}) = -1$$

Correction de l'Exercice 7

Question 1 : Montrer que la suite (x_n) est bornée.

Pour montrer que la suite (x_n) est bornée, nous devons trouver une constante $M > 0$ telle que $|x_n| \leq M$ pour tout $n \in \mathbb{N}$. La suite (x_n) est définie par $x_n = \cos\left(\frac{n\pi}{3}\right) + \frac{(-1)^n}{n+1}$.

Nous allons majorer séparément les valeurs absolues de chaque terme.

Pour le premier terme, $\cos\left(\frac{n\pi}{3}\right)$: La fonction cosinus est bornée, et pour tout $t \in \mathbb{R}$, on a $|\cos(t)| \leq 1$. Ainsi, pour tout $n \in \mathbb{N}$, nous avons :

$$\left|\cos\left(\frac{n\pi}{3}\right)\right| \leq 1$$

Pour le second terme, $\frac{(-1)^n}{n+1}$: Pour tout $n \in \mathbb{N}$, nous avons :

$$\left|\frac{(-1)^n}{n+1}\right| = \frac{|(-1)^n|}{|n+1|} = \frac{1}{n+1}$$

Puisque $n \in \mathbb{N}$, $n \geq 0$, donc $n+1 \geq 1$. Par conséquent, $\frac{1}{n+1} \leq 1$ pour tout $n \in \mathbb{N}$. Ainsi, pour tout $n \in \mathbb{N}$, nous avons :

$$\left|\frac{(-1)^n}{n+1}\right| \leq 1$$

Maintenant, nous utilisons l'inégalité triangulaire pour x_n :

$$|x_n| = \left|\cos\left(\frac{n\pi}{3}\right) + \frac{(-1)^n}{n+1}\right| \leq \left|\cos\left(\frac{n\pi}{3}\right)\right| + \left|\frac{(-1)^n}{n+1}\right|$$

En utilisant les majorations établies précédemment :

$$|x_n| \leq 1 + 1 = 2$$

Donc, pour tout $n \in \mathbb{N}$, $|x_n| \leq 2$. La suite (x_n) est donc bornée par $M = 2$.

Question 2 : Déterminer l'ensemble A de toutes les valeurs d'adhérence de (x_n) . Justifier rigoureusement votre réponse.

Soit $x_n = u_n + v_n$, où $u_n = \cos\left(\frac{n\pi}{3}\right)$ et $v_n = \frac{(-1)^n}{n+1}$.

Commençons par analyser la suite (v_n) :

$$v_n = \frac{(-1)^n}{n+1}$$

Nous avons $\lim_{n \rightarrow \infty} |v_n| = \lim_{n \rightarrow \infty} \frac{1}{n+1} = 0$. Par conséquent, $\lim_{n \rightarrow \infty} v_n = 0$.

Maintenant, analysons la suite (u_n) :

$$u_n = \cos\left(\frac{n\pi}{3}\right)$$

La suite (u_n) est une suite périodique. La période de $\cos(k\theta)$ est $2\pi/\theta$. Ici $\theta = \pi/3$, donc la période est $2\pi/(\pi/3) = 6$. Les valeurs prises par u_n sont : - $u_0 = \cos(0) = 1$ - $u_1 = \cos(\pi/3) = 1/2$ - $u_2 = \cos(2\pi/3) = -1/2$ - $u_3 = \cos(\pi) = -1$ - $u_4 = \cos(4\pi/3) = -1/2$ - $u_5 = \cos(5\pi/3) = 1/2$ - $u_6 = \cos(2\pi) = 1 = u_0$. L'ensemble des valeurs prises par la suite (u_n) est $S_u = \{1, 1/2, -1/2, -1\}$. Puisque (u_n) est une suite périodique, l'ensemble de ses valeurs d'adhérence est précisément l'ensemble des valeurs qu'elle prend, c'est-à-dire S_u .

Soit L une valeur d'adhérence de (x_n) . Par définition, il existe une sous-suite $(x_{\phi(k)})_{k \in \mathbb{N}}$ qui converge vers L .

$$x_{\phi(k)} = u_{\phi(k)} + v_{\phi(k)}$$

Puisque $\lim_{n \rightarrow \infty} v_n = 0$, il s'ensuit que $\lim_{k \rightarrow \infty} v_{\phi(k)} = 0$. Donc, si $x_{\phi(k)} \rightarrow L$, alors $u_{\phi(k)} = x_{\phi(k)} - v_{\phi(k)} \rightarrow L - 0 = L$. Ceci signifie que toute valeur d'adhérence de (x_n) doit être une valeur d'adhérence de (u_n) . Par conséquent, $A \subseteq S_u$.

Réciproquement, montrons que chaque élément de S_u est une valeur d'adhérence de (x_n) . Soit $L \in S_u$. Par définition de S_u , il existe un indice $j \in \{0, 1, 2, 3, 4, 5\}$ tel que $u_j = L$. Considérons la sous-suite $(x_{6k+j})_{k \in \mathbb{N}}$. Pour cette sous-suite, le terme u_{6k+j} est :

$$u_{6k+j} = \cos\left(\frac{(6k+j)\pi}{3}\right) = \cos\left(2k\pi + \frac{j\pi}{3}\right) = \cos\left(\frac{j\pi}{3}\right) = u_j = L$$

Le terme v_{6k+j} est :

$$v_{6k+j} = \frac{(-1)^{6k+j}}{6k+j+1} = \frac{(-1)^j}{6k+j+1}$$

Puisque $\lim_{k \rightarrow \infty} (6k+j+1) = \infty$, nous avons $\lim_{k \rightarrow \infty} v_{6k+j} = 0$. Par conséquent, la sous-suite (x_{6k+j}) converge vers L :

$$\lim_{k \rightarrow \infty} x_{6k+j} = \lim_{k \rightarrow \infty} (u_{6k+j} + v_{6k+j}) = L + 0 = L$$

Ceci montre que chaque élément de S_u est une valeur d'adhérence de (x_n) . Par conséquent, $S_u \subseteq A$.

En combinant les deux inclusions ($A \subseteq S_u$ et $S_u \subseteq A$), nous concluons que l'ensemble des valeurs d'adhérence de (x_n) est $A = S_u$.

$$A = \left\{1, \frac{1}{2}, -\frac{1}{2}, -1\right\}$$

Question 3 : La suite (x_n) converge-t-elle ?

Une suite de nombres réels converge si et seulement si elle est bornée et possède une unique valeur d'adhérence. D'après la question 1, la suite (x_n) est bornée. D'après la question 2, l'ensemble des valeurs d'adhérence de (x_n) est $A = \{1, 1/2, -1/2, -1\}$. Cet ensemble contient quatre éléments distincts. Puisque la suite (x_n) possède plus d'une valeur d'adhérence, elle ne converge pas.

Question 4 : Soit $(y_N)_{N \in \mathbb{N}^*}$ la suite définie par $y_N = \frac{1}{N} \sum_{k=0}^{N-1} x_k$. La suite (y_N) converge-t-elle ? Si oui, quelle est sa limite ?

La suite (y_N) est la moyenne de Cesaro de la suite (x_n) . Nous avons $x_k = \cos\left(\frac{k\pi}{3}\right) + \frac{(-1)^k}{k+1}$. Donc, $y_N = \frac{1}{N} \sum_{k=0}^{N-1} \left(\cos\left(\frac{k\pi}{3}\right) + \frac{(-1)^k}{k+1}\right)$. Nous pouvons séparer la somme en deux parties :

$$y_N = \frac{1}{N} \sum_{k=0}^{N-1} \cos\left(\frac{k\pi}{3}\right) + \frac{1}{N} \sum_{k=0}^{N-1} \frac{(-1)^k}{k+1}$$

Considérons le second terme : $T_N^{(2)} = \frac{1}{N} \sum_{k=0}^{N-1} \frac{(-1)^k}{k+1}$. La série $\sum_{k=0}^{\infty} \frac{(-1)^k}{k+1}$ est une série alternée. Soit $a_k = \frac{1}{k+1}$. La suite (a_k) est positive, décroissante et tend vers 0. D'après le critère de Leibniz, la série $\sum_{k=0}^{\infty} (-1)^k a_k$ converge. Sa somme est $\ln(2)$. Soit $S = \sum_{k=0}^{\infty} \frac{(-1)^k}{k+1}$. Alors $\sum_{k=0}^{N-1} \frac{(-1)^k}{k+1}$ est la N -ième somme partielle de cette série convergente. Notons $P_N = \sum_{k=0}^{N-1} \frac{(-1)^k}{k+1}$. Nous savons que $\lim_{N \rightarrow \infty} P_N = S = \ln(2)$. Le terme $T_N^{(2)}$ est alors $\frac{P_N}{N}$. Puisque P_N converge vers une limite finie S , et $N \rightarrow \infty$, nous avons :

$$\lim_{N \rightarrow \infty} T_N^{(2)} = \lim_{N \rightarrow \infty} \frac{P_N}{N} = 0$$

Considérons le premier terme : $T_N^{(1)} = \frac{1}{N} \sum_{k=0}^{N-1} \cos\left(\frac{k\pi}{3}\right)$. La suite $u_k = \cos\left(\frac{k\pi}{3}\right)$ est périodique de période 6. Calculons la somme des termes sur une période complète :

$$P = \sum_{k=0}^5 \cos\left(\frac{k\pi}{3}\right) = \cos(0) + \cos(\pi/3) + \cos(2\pi/3) + \cos(\pi) + \cos(4\pi/3) + \cos(5\pi/3)$$

$$P = 1 + \frac{1}{2} - \frac{1}{2} - 1 - \frac{1}{2} + \frac{1}{2} = 0$$

Soit $N \in \mathbb{N}^*$. Nous pouvons écrire $N = 6q + r$, où $q = \lfloor N/6 \rfloor$ est le quotient et $r = N \pmod{6}$ est le reste, avec $0 \leq r < 6$. La somme $\sum_{k=0}^{N-1} \cos\left(\frac{k\pi}{3}\right)$ peut s'écrire comme :

$$\sum_{k=0}^{N-1} u_k = \sum_{k=0}^{6q-1} u_k + \sum_{k=6q}^{N-1} u_k$$

La première partie est la somme de q périodes complètes :

$$\sum_{k=0}^{6q-1} u_k = q \cdot P = q \cdot 0 = 0$$

La seconde partie est la somme des r premiers termes de la période suivante (ou les r derniers termes de la somme si $N - 1$ est le dernier indice) :

$$\sum_{k=6q}^{N-1} u_k = \sum_{j=0}^{r-1} u_{6q+j} = \sum_{j=0}^{r-1} \cos\left(\frac{(6q+j)\pi}{3}\right) = \sum_{j=0}^{r-1} \cos\left(2q\pi + \frac{j\pi}{3}\right) = \sum_{j=0}^{r-1} \cos\left(\frac{j\pi}{3}\right)$$

Cette somme est une somme finie de r termes, où $r \in \{0, 1, 2, 3, 4, 5\}$. La valeur maximale de cette somme est $1 + 1/2 = 3/2$ (pour $r = 2$). La valeur minimale est $-1/2$ (pour $r = 5$). En général, cette somme est bornée. Soit $M_S = \max_{0 \leq r < 6} \left| \sum_{j=0}^{r-1} \cos\left(\frac{j\pi}{3}\right) \right|$. On a $M_S = 3/2$. Donc, $\left| \sum_{k=0}^{N-1} \cos\left(\frac{k\pi}{3}\right) \right| = \left| \sum_{j=0}^{r-1} \cos\left(\frac{j\pi}{3}\right) \right| \leq M_S$. Par conséquent, $T_N^{(1)} = \frac{1}{N} \sum_{k=0}^{N-1} \cos\left(\frac{k\pi}{3}\right)$ satisfait :

$$\left| T_N^{(1)} \right| \leq \frac{M_S}{N}$$

Puisque M_S est une constante finie et $N \rightarrow \infty$, nous avons :

$$\lim_{N \rightarrow \infty} T_N^{(1)} = 0$$

En combinant les limites des deux termes :

$$\lim_{N \rightarrow \infty} y_N = \lim_{N \rightarrow \infty} T_N^{(1)} + \lim_{N \rightarrow \infty} T_N^{(2)} = 0 + 0 = 0$$

La suite (y_N) converge, et sa limite est 0.

Conclusion : La suite (y_N) converge vers 0.

Énoncé de l'Exercice 8

Soit $(u_n)_{n \in \mathbb{N}}$ une suite bornée de nombres réels. On note $A(u)$ l'ensemble de ses valeurs d'adhérence.

1. Démontrer que $A(u)$ est un ensemble non vide et compact de $\mathbb{R}$.

2. On suppose de plus qu'il existe une fonction $f : \mathbb{R} \rightarrow \mathbb{R}$ continue telle que $u_{n+1} = f(u_n)$ pour tout $n \in \mathbb{N}$.
a. Montrer que si $x \in A(u)$, alors $f(x) \in A(u)$. b. En déduire que si la suite (u_n) converge, sa limite est un point fixe de f . c. Montrer que si $A(u)$ est un singleton, alors la suite (u_n) converge.

3. On rappelle que la limite supérieure de (u_n) est définie par $\limsup_{n \rightarrow \infty} u_n = \lim_{n \rightarrow \infty} (\sup_{k \geq n} u_k)$ et la limite inférieure par $\liminf_{n \rightarrow \infty} u_n = \lim_{n \rightarrow \infty} (\inf_{k \geq n} u_k)$. Démontrer que $\sup A(u) = \limsup_{n \rightarrow \infty} u_n$ et $\inf A(u) = \liminf_{n \rightarrow \infty} u_n$.

Correction de l'Exercice 8

Question 1 : Démontrer que $A(u)$ est un ensemble non vide et compact de $\mathbb{R}$.

Pour démontrer que $A(u)$ est compact, nous devons montrer qu'il est fermé et borné dans $\mathbb{R}$. De plus, nous devons montrer qu'il est non vide.

1. $A(u)$ est non vide : La suite $(u_n)_{n \in \mathbb{N}}$ est bornée par hypothèse. D'après le théorème de Bolzano-Weierstrass, toute suite bornée de nombres réels admet au moins une sous-suite convergente. Soit $(u_{\phi(k)})_{k \in \mathbb{N}}$ une telle sous-suite convergente. Sa limite $L = \lim_{k \rightarrow \infty} u_{\phi(k)}$ est, par définition, une valeur d'adhérence de la suite (u_n) . Par conséquent, $L \in A(u)$, ce qui prouve que $A(u)$ est non vide.

2. $A(u)$ est borné : Puisque la suite (u_n) est bornée, il existe un intervalle fermé borné $[m, M]$ tel que $u_n \in [m, M]$ pour tout $n \in \mathbb{N}$. Soit $x \in A(u)$. Par définition, il existe une sous-suite $(u_{\phi(k)})$ de (u_n) qui converge vers x . Puisque $u_{\phi(k)} \in [m, M]$ pour tout k , et que l'intervalle $[m, M]$ est fermé, la limite x doit également appartenir à $[m, M]$. En effet, si $x \notin [m, M]$, alors il existerait un $\epsilon > 0$ tel que l'intervalle $(x - \epsilon, x + \epsilon)$ ne contiendrait aucun point de $[m, M]$, ce qui contredirait la convergence de $(u_{\phi(k)})$ vers x . Ainsi, tout élément de $A(u)$ appartient à $[m, M]$. Cela signifie que $A(u)$ est borné.

3. $A(u)$ est fermé : Pour montrer que $A(u)$ est fermé, nous allons montrer que toute suite convergente d'éléments de $A(u)$ a sa limite dans $A(u)$. Soit $(x_j)_{j \in \mathbb{N}}$ une suite d'éléments de $A(u)$ qui converge vers un réel x . Nous devons montrer que $x \in A(u)$. Puisque $x_j \in A(u)$ pour chaque j , il existe une sous-suite $(u_{\phi_j(k)})_{k \in \mathbb{N}}$ de (u_n) qui converge vers x_j . Nous allons construire une sous-suite de (u_n) qui converge vers x . Pour chaque $j \in \mathbb{N}$, puisque $u_{\phi_j(k)} \rightarrow x_j$ lorsque $k \rightarrow \infty$, il existe un indice k_j tel que pour tout $k \geq k_j$, $|u_{\phi_j(k)} - x_j| < \frac{1}{j+1}$. De plus, puisque $x_j \rightarrow x$ lorsque $j \rightarrow \infty$, il existe un indice J tel que pour tout $j \geq J$, $|x_j - x| < \frac{1}{j+1}$. Nous pouvons construire une sous-suite $(u_{\psi(j)})$ de (u_n) qui converge vers x de la manière suivante : Pour chaque $j \in \mathbb{N}$, choisissons un indice $n_j = \phi_j(k_j)$ tel que $n_j > n_{j-1}$ (pour assurer que la suite d'indices est strictement croissante) et tel que $|u_{n_j} - x_j| < \frac{1}{j+1}$. Alors, pour ce n_j , nous avons : $|u_{n_j} - x| = |u_{n_j} - x_j + x_j - x| \leq |u_{n_j} - x_j| + |x_j - x| < \frac{1}{j+1} + \frac{1}{j+1} < \frac{\epsilon}{2}$. Puisque $x_j \rightarrow x$, pour tout $\epsilon > 0$, il existe J_1 tel que pour $j \geq J_1$, $|x_j - x| < \epsilon/2$. Puisque $u_{\phi_j(k)} \rightarrow x_j$, pour tout j , il existe K_j tel que pour $k \geq K_j$, $|u_{\phi_j(k)} - x_j| < \epsilon/2$. Nous pouvons choisir une sous-suite $(u_{\psi(j)})$ de (u_n) telle que pour chaque j , $u_{\psi(j)}$ est un terme de la sous-suite $(u_{\phi_j(k)})$ et $\psi(j) > \psi(j-1)$ (pour assurer que c'est bien une sous-suite) et $|u_{\psi(j)} - x_j| < \frac{1}{j+1}$. Alors, pour j suffisamment grand (plus grand que J_1 et tel que $\frac{1}{j+1} < \epsilon/2$), nous avons : $|u_{\psi(j)} - x| \leq |u_{\psi(j)} - x_j| + |x_j - x| < \frac{1}{j+1} + \frac{\epsilon}{2} < \frac{\epsilon}{2} + \frac{\epsilon}{2} = \epsilon$. Ainsi, la sous-suite $(u_{\psi(j)})$ converge vers x . Par conséquent, $x \in A(u)$. Donc $A(u)$ est fermé.

Conclusion : Puisque $A(u)$ est non vide, borné et fermé dans $\mathbb{R}$, il est compact d'après le théorème de Heine-Borel.

Question 2 : Propriétés des suites récurrentes.

2.a. Montrer que si $x \in A(u)$, alors $f(x) \in A(u)$. Soit $x \in A(u)$. Par définition, il existe une sous-suite $(u_{\phi(k)})_{k \in \mathbb{N}}$ de (u_n) qui converge vers x . Puisque f est une fonction continue, et que $u_{\phi(k)} \rightarrow x$ lorsque $k \rightarrow \infty$, la suite $(f(u_{\phi(k)}))_{k \in \mathbb{N}}$ converge vers $f(x)$. Or, par la relation de récurrence $u_{n+1} = f(u_n)$, nous avons $f(u_{\phi(k)}) = u_{\phi(k)+1}$. Donc, la suite $(u_{\phi(k)+1})_{k \in \mathbb{N}}$ est une sous-suite de (u_n) (car $\phi(k) + 1$ est une suite d'indices strictement croissante si $\phi(k)$ l'est) qui converge vers $f(x)$. Par conséquent, $f(x)$ est une valeur d'adhérence de la suite (u_n) , ce qui signifie $f(x) \in A(u)$.

2.b. En déduire que si la suite (u_n) converge, sa limite est un point fixe de f . Supposons que la suite (u_n) converge vers une limite L . Si (u_n) converge vers L , alors l'ensemble de ses valeurs d'adhérence est le singleton $A(u) = \{L\}$. En effet, toute sous-suite de (u_n) converge vers L . D'après la question 2.a, si $x \in A(u)$, alors $f(x) \in A(u)$. Puisque $L \in A(u)$, il s'ensuit que $f(L) \in A(u)$. Comme $A(u) = \{L\}$, la seule possibilité est que $f(L) = L$. Par conséquent, L est un point fixe de f .

2.c. Montrer que si $A(u)$ est un singleton, alors la suite (u_n) converge. Supposons que $A(u) = \{L\}$ pour un certain réel L . Nous voulons montrer que $\lim_{n \rightarrow \infty} u_n = L$. Puisque la suite (u_n) est bornée, elle est contenue dans un intervalle fermé borné $[m, M]$. Supposons par l'absurde que (u_n) ne converge pas vers L . Cela signifie qu'il existe un $\epsilon_0 > 0$ tel que pour tout $N \in \mathbb{N}$, il existe un $n \geq N$ tel que $|u_n - L| \geq \epsilon_0$. Autrement dit, il existe une infinité de termes de la suite (u_n) qui se trouvent en dehors de l'intervalle ouvert $(L - \epsilon_0, L + \epsilon_0)$. Ces termes forment une sous-suite $(u_{\psi(k)})$ telle que $u_{\psi(k)} \notin (L - \epsilon_0, L + \epsilon_0)$ pour tout k . Cette sous-suite $(u_{\psi(k)})$ est elle-même bornée (car (u_n) est bornée). D'après le théorème de Bolzano-Weierstrass, cette sous-suite $(u_{\psi(k)})$ admet une sous-suite convergente $(u_{\psi(\chi(j))})$. Soit $L' = \lim_{j \rightarrow \infty} u_{\psi(\chi(j))}$. Par définition, L' est une valeur d'adhérence de (u_n) , donc $L' \in A(u)$. Cependant, puisque $u_{\psi(\chi(j))} \notin (L - \epsilon_0, L + \epsilon_0)$ pour tout j , la limite L' doit également satisfaire $|L' - L| \geq \epsilon_0$. En effet, si $|L' - L| < \epsilon_0$, alors pour j suffisamment grand, $u_{\psi(\chi(j))}$ serait dans $(L - \epsilon_0, L + \epsilon_0)$, ce qui est une contradiction. Donc $L' \neq L$. Ceci contredit l'hypothèse que $A(u) = \{L\}$ est un singleton. Par conséquent, l'hypothèse que (u_n) ne converge pas vers L est fausse. La suite (u_n) converge donc vers L .

Question 3 : Relation entre $A(u)$ et les limites supérieure et inférieure.

Soit $L^* = \limsup_{n \rightarrow \infty} u_n$ et $L_* = \liminf_{n \rightarrow \infty} u_n$.

1. Démontrons que $L^* \in A(u)$ et $L_* \in A(u)$. Par définition, $L^* = \lim_{n \rightarrow \infty} v_n$, où $v_n = \sup_{k \geq n} u_k$. La suite (v_n) est décroissante et minorée (par n'importe quel minorant de (u_n)), donc elle converge. Pour montrer que $L^* \in A(u)$, nous devons construire une sous-suite de (u_n) qui converge vers L^* . Pour tout $\epsilon > 0$ et pour tout $N \in \mathbb{N}$, il existe $k \geq N$ tel que $u_k > L^* - \epsilon$. (Si ce n'était pas le cas, alors pour un certain $\epsilon_0 > 0$ et N_0 , pour tout $k \geq N_0$, $u_k \leq L^* - \epsilon_0$, ce qui impliquerait $v_{N_0} \leq L^* - \epsilon_0$, contredisant $v_n \rightarrow L^*$). De plus, pour tout $\epsilon > 0$, il existe $N_0 \in \mathbb{N}$ tel que pour tout $n \geq N_0$, $v_n < L^* + \epsilon$. Cela implique que pour tout $n \geq N_0$ et tout $k \geq n$, $u_k \leq v_n < L^* + \epsilon$. Combinons ces deux propriétés pour construire la sous-suite : Pour $\epsilon = 1$, il existe $n_1 \geq 0$ tel que $L^* - 1 < u_{n_1} < L^* + 1$. Pour $\epsilon = 1/2$, il existe $n_2 > n_1$ tel que $L^* - 1/2 < u_{n_2} < L^* + 1/2$. En général, pour $\epsilon = 1/j$, il existe $n_j > n_{j-1}$ tel que $L^* - 1/j < u_{n_j} < L^* + 1/j$. La suite $(u_{n_j})_{j \in \mathbb{N}}$ est une sous-suite de (u_n) et, par le théorème des gendarmes, elle converge vers L^* . Donc $L^* \in A(u)$.

De manière similaire, pour $L_* = \liminf_{n \rightarrow \infty} u_n = \lim_{n \rightarrow \infty} w_n$, où $w_n = \inf_{k \geq n} u_k$. La suite (w_n) est croissante et majorée, donc elle converge. Pour tout $\epsilon > 0$ et pour tout $N \in \mathbb{N}$, il existe $k \geq N$ tel que $u_k < L_* + \epsilon$. De plus, pour tout $\epsilon > 0$, il existe $N_0 \in \mathbb{N}$ tel que pour tout $n \geq N_0$, $w_n > L_* - \epsilon$. Cela implique que pour tout $n \geq N_0$ et tout $k \geq n$, $u_k \geq w_n > L_* - \epsilon$. En utilisant une construction similaire à celle pour L^* , nous pouvons construire une sous-suite de (u_n) qui converge vers L_* . Donc $L_* \in A(u)$.

2. Démontrons que pour tout $x \in A(u)$, $L_* \leq x \leq L^*$. Soit $x \in A(u)$. Il existe une sous-suite $(u_{\phi(k)})$ qui converge vers x . Par définition de $v_n = \sup_{j \geq n} u_j$, nous avons $u_j \leq v_n$ pour tout $j \geq n$. En particulier, pour tout k tel que $\phi(k) \geq n$, nous avons $u_{\phi(k)} \leq v_n$. En passant à la limite lorsque $k \rightarrow \infty$ (et donc $\phi(k) \rightarrow \infty$), nous obtenons $x \leq v_n$ pour tout n . Ensuite, en passant à la limite lorsque $n \rightarrow \infty$, nous obtenons $x \leq \lim_{n \rightarrow \infty} v_n = L^*$.

De même, par définition de $w_n = \inf_{j \geq n} u_j$, nous avons $u_j \geq w_n$ pour tout $j \geq n$. En particulier, pour tout k tel que $\phi(k) \geq n$, nous avons $u_{\phi(k)} \geq w_n$. En passant à la limite lorsque $k \rightarrow \infty$, nous obtenons $x \geq w_n$ pour tout n . Ensuite, en passant à la limite lorsque $n \rightarrow \infty$, nous obtenons $x \geq \lim_{n \rightarrow \infty} w_n = L_*$. Ainsi, pour tout $x \in A(u)$, nous avons $L_* \leq x \leq L^*$.

3. Conclusion : Nous avons montré que $L^* \in A(u)$ et que L^* est un majorant de $A(u)$. Par définition du supremum, L^* est le plus petit des majorants de $A(u)$. Par conséquent, $\sup A(u) = L^* = \limsup_{n \rightarrow \infty} u_n$. De même, nous avons montré que $L_* \in A(u)$ et que L_* est un minorant de $A(u)$. Par définition de l'infimum, L_* est le plus grand des minorants de $A(u)$. Par conséquent, $\inf A(u) = L_* = \liminf_{n \rightarrow \infty} u_n$.

Ceci conclut la démonstration.

◻ ◻ ◻

Soit la suite $(x_n)_{n \in \mathbb{N}}$ définie par $x_0 \in [0, 1]$ et pour tout $n \geq 0$ par les relations de récurrence suivantes :

$$\begin{cases} x_{2n+1} = x_{2n}^2 \\ x_{2n+2} = 1 - x_{2n+1} \end{cases}$$

1. Montrer que si $x_0 \in [0, 1]$, alors pour tout $n \in \mathbb{N}$, $x_n \in [0, 1]$. 2. En déduire que la suite (x_n) admet au moins une valeur d'adhérence. 3. On définit les sous-suites $(y_n)_{n \in \mathbb{N}}$ et $(z_n)_{n \in \mathbb{N}}$ par $y_n = x_{2n}$ et $z_n = x_{2n+1}$. a. Établir les relations de récurrence pour (y_n) et (z_n) . b. Déterminer les points fixes de ces récurrences dans l'intervalle $[0, 1]$. 4. Soit S l'ensemble des valeurs d'adhérence de la suite (x_n) . a. Montrer que S est l'union des ensembles de valeurs d'adhérence de (y_n) et (z_n) . b. Pour $x_0 = \frac{\sqrt{5}-1}{2}$, déterminer l'ensemble S . Justifier rigoureusement. c. Pour $x_0 = 0$, déterminer l'ensemble S . Justifier rigoureusement.

Correction de l'exercice 9

1. Montrer que si $x_0 \in [0, 1]$, alors pour tout $n \in \mathbb{N}$, $x_n \in [0, 1]$.

Nous allons procéder par récurrence. **Initialisation** : Pour $n = 0$, $x_0 \in [0, 1]$ par hypothèse.

Hérédité : Supposons que pour un certain $n \in \mathbb{N}$, $x_{2n} \in [0, 1]$. Alors, la première relation de récurrence donne $x_{2n+1} = x_{2n}^2$. Puisque $x_{2n} \in [0, 1]$, on a $0 \leq x_{2n} \leq 1$. En élevant au carré, on obtient $0^2 \leq x_{2n}^2 \leq 1^2$, ce qui implique $0 \leq x_{2n+1} \leq 1$. Donc $x_{2n+1} \in [0, 1]$.

Ensuite, la deuxième relation de récurrence donne $x_{2n+2} = 1 - x_{2n+1}$. Puisque $x_{2n+1} \in [0, 1]$, on a $0 \leq x_{2n+1} \leq 1$. En multipliant par -1 , on obtient $-1 \leq -x_{2n+1} \leq 0$. En ajoutant 1, on obtient $1 - 1 \leq 1 - x_{2n+1} \leq 1 + 0$, ce qui implique $0 \leq x_{2n+2} \leq 1$. Donc $x_{2n+2} \in [0, 1]$.

Ainsi, si $x_{2n} \in [0, 1]$, alors $x_{2n+1} \in [0, 1]$ et $x_{2n+2} \in [0, 1]$. Cela signifie que si un terme d'indice pair est dans $[0, 1]$, les deux termes suivants (un impair et un pair) sont également dans $[0, 1]$. Par conséquent, par récurrence, tous les termes de la suite (x_n) sont dans $[0, 1]$.

Conclusion : Pour tout $n \in \mathbb{N}$, $x_n \in [0, 1]$.

2. En déduire que la suite (x_n) admet au moins une valeur d'adhérence.

La suite (x_n) est une suite de nombres réels. D'après la question 1, nous avons montré que pour tout $n \in \mathbb{N}$, $x_n \in [0, 1]$. Cela signifie que la suite (x_n) est bornée, car tous ses termes sont compris entre 0 et 1. Le théorème de Bolzano-Weierstrass stipule que toute suite réelle bornée admet au moins une valeur d'adhérence. Par conséquent, la suite (x_n) admet au moins une valeur d'adhérence.

3. On définit les sous-suites $(y_n)_{n \in \mathbb{N}}$ et $(z_n)_{n \in \mathbb{N}}$ par $y_n = x_{2n}$ et $z_n = x_{2n+1}$.

a. Établir les relations de récurrence pour (y_n) et (z_n) .

Pour la suite (y_n) : $y_n = x_{2n}$. $y_{n+1} = x_{2(n+1)} = x_{2n+2}$. D'après la deuxième relation de récurrence de (x_n) , nous avons $x_{2n+2} = 1 - x_{2n+1}$. D'après la première relation de récurrence de (x_n) , nous avons $x_{2n+1} = x_{2n}^2$. En substituant x_{2n+1} dans l'expression de x_{2n+2} , nous obtenons $x_{2n+2} = 1 - x_{2n}^2$. Puisque $y_n = x_{2n}$ et $y_{n+1} = x_{2n+2}$, la relation de récurrence pour (y_n) est :

$$y_{n+1} = 1 - y_n^2$$

Le terme initial est $y_0 = x_0$.

Pour la suite (z_n) : $z_n = x_{2n+1}$. $z_{n+1} = x_{2(n+1)+1} = x_{2n+3}$. D'après la première relation de récurrence de (x_n) , nous avons $x_{2n+3} = x_{2n+2}^2$. D'après la deuxième relation de récurrence de (x_n) , nous avons $x_{2n+2} = 1 - x_{2n+1}$. En substituant x_{2n+2} dans l'expression de x_{2n+3} , nous obtenons $x_{2n+3} = (1 - x_{2n+1})^2$. Puisque $z_n = x_{2n+1}$ et $z_{n+1} = x_{2n+3}$, la relation de récurrence pour (z_n) est :

$$z_{n+1} = (1 - z_n)^2$$

Le terme initial est $z_0 = x_1 = x_0^2$.

b. Déterminer les points fixes de ces récurrences dans $[0, 1]$.

Pour la récurrence $y_{n+1} = 1 - y_n^2$: Un point fixe L_y satisfait $L_y = 1 - L_y^2$. Ceci est équivalent à $L_y^2 + L_y - 1 = 0$.

Les solutions de cette équation quadratique sont données par la formule $L_y = \frac{-1 \pm \sqrt{1^2 - 4(1)(-1)}}{2(1)} = \frac{-1 \pm \sqrt{1+4}}{2} = \frac{-1 \pm \sqrt{5}}{2}$. Nous cherchons les points fixes dans l'intervalle $[0, 1]$. La solution $L_y = \frac{-1 - \sqrt{5}}{2}$ est négative (environ -1.618), donc elle n'est pas dans $[0, 1]$. La solution $L_y = \frac{-1 + \sqrt{5}}{2}$ est positive (environ 0.618). Puisque $\sqrt{5} \approx 2.236$, $0 < \frac{-1 + \sqrt{5}}{2} < 1$. Donc, le seul point fixe de la récurrence (y_n) dans $[0, 1]$ est $\alpha = \frac{\sqrt{5}-1}{2}$.

Pour la récurrence $z_{n+1} = (1 - z_n)^2$: Un point fixe L_z satisfait $L_z = (1 - L_z)^2$. Ceci est équivalent à $L_z = 1 - 2L_z + L_z^2$. Réarrangeant les termes, nous obtenons $L_z^2 - 3L_z + 1 = 0$. Les solutions de cette équation

quadratique sont données par la formule $L_z = \frac{-(-3) \pm \sqrt{(-3)^2 - 4(1)(1)}}{2(1)} = \frac{3 \pm \sqrt{9-4}}{2} = \frac{3 \pm \sqrt{5}}{2}$. Nous cherchons les points fixes dans l'intervalle $[0, 1]$. La solution $L_z = \frac{3+\sqrt{5}}{2}$ est supérieure à 1 (environ 2.618), donc elle n'est pas dans $[0, 1]$. La solution $L_z = \frac{3-\sqrt{5}}{2}$ est positive (environ 0.382). Puisque $\sqrt{5} \approx 2.236$, $0 < \frac{3-\sqrt{5}}{2} < 1$. Donc, le seul point fixe de la récurrence (z_n) dans $[0, 1]$ est $\beta = \frac{3-\sqrt{5}}{2}$.

4. Soit S l'ensemble des valeurs d'adhérence de la suite (x_n) .

a. Montrer que S est l'union des ensembles de valeurs d'adhérence de (y_n) et (z_n) .

Soit S_y l'ensemble des valeurs d'adhérence de (y_n) et S_z l'ensemble des valeurs d'adhérence de (z_n) . Par définition, $y_n = x_{2n}$ est la sous-suite des termes d'indices pairs de (x_n) , et $z_n = x_{2n+1}$ est la sous-suite des termes d'indices impairs de (x_n) .

1. **Montrons que $S_y \cup S_z \subseteq S$:** Soit $L \in S_y$. Par définition, il existe une sous-suite (y_{n_k}) de (y_n) telle que $y_{n_k} \rightarrow L$. Puisque $y_{n_k} = x_{2n_k}$, la suite (x_{2n_k}) est une sous-suite de (x_n) qui converge vers L . Donc L est une valeur d'adhérence de (x_n) , ce qui signifie $L \in S$. De même, si $L \in S_z$, il existe une sous-suite (z_{n_k}) de (z_n) telle que $z_{n_k} \rightarrow L$. Puisque $z_{n_k} = x_{2n_k+1}$, la suite (x_{2n_k+1}) est une sous-suite de (x_n) qui converge vers L . Donc L est une valeur d'adhérence de (x_n) , ce qui signifie $L \in S$. Par conséquent, $S_y \cup S_z \subseteq S$.

2. **Montrons que $S \subseteq S_y \cup S_z$:** Soit $L \in S$. Par définition, il existe une sous-suite (x_{k_j}) de (x_n) telle que $x_{k_j} \rightarrow L$. La suite des indices (k_j) peut contenir une infinité d'indices pairs, une infinité d'indices impairs, ou une infinité d'indices d'un seul type. * **Cas 1 :** La sous-suite (x_{k_j}) contient une infinité de termes d'indices pairs. Alors il existe une sous-sous-suite $(x_{k_{j_m}})$ où tous les k_{j_m} sont pairs. Soit $k_{j_m} = 2n_m$. Alors (x_{2n_m}) est une sous-suite de (y_n) qui converge vers L . Donc $L \in S_y$. * **Cas 2 :** La sous-suite (x_{k_j}) contient une infinité de termes d'indices impairs. Alors il existe une sous-sous-suite $(x_{k_{j_m}})$ où tous les k_{j_m} sont impairs. Soit $k_{j_m} = 2n_m + 1$. Alors (x_{2n_m+1}) est une sous-suite de (z_n) qui converge vers L . Donc $L \in S_z$. * **Cas 3 :** La sous-suite (x_{k_j}) contient un nombre fini d'indices pairs et un nombre fini d'indices impairs. Ceci est impossible car la suite (k_j) est infinie. Elle doit contenir une infinité d'indices pairs ou une infinité d'indices impairs (ou les deux).

Dans tous les cas, $L \in S_y$ ou $L \in S_z$. Par conséquent, $S \subseteq S_y \cup S_z$.

Conclusion : L'ensemble des valeurs d'adhérence de (x_n) est l'union des ensembles de valeurs d'adhérence de (y_n) et (z_n) , c'est-à-dire $S = S_y \cup S_z$.

b. Pour $x_0 = \frac{\sqrt{5}-1}{2}$, déterminer l'ensemble S . Justifier rigoureusement.

Soit $x_0 = \alpha = \frac{\sqrt{5}-1}{2}$.

Pour la suite (y_n) : Nous avons $y_0 = x_0 = \alpha$. La relation de récurrence est $y_{n+1} = 1 - y_n^2$. D'après la question 3b, α est un point fixe de cette récurrence. Donc, $y_1 = 1 - y_0^2 = 1 - \alpha^2 = \alpha$. Par récurrence immédiate, $y_n = \alpha$ pour tout $n \in \mathbb{N}$. La suite (y_n) est la suite constante $(\alpha, \alpha, \alpha, \dots)$. L'ensemble des valeurs d'adhérence de (y_n) est $S_y = \{\alpha\}$.

Pour la suite (z_n) : Nous avons $z_0 = x_0^2 = \alpha^2$. Calculons α^2 : $\alpha^2 = \left(\frac{\sqrt{5}-1}{2}\right)^2 = \frac{(\sqrt{5})^2 - 2\sqrt{5} + 1}{4} = \frac{5 - 2\sqrt{5} + 1}{4} = \frac{6 - 2\sqrt{5}}{4} = \frac{3 - \sqrt{5}}{2}$. D'après la question 3b, $\beta = \frac{3-\sqrt{5}}{2}$ est le point fixe de la récurrence $z_{n+1} = (1 - z_n)^2$. Donc $z_0 = \beta$. Puisque β est un point fixe, $z_1 = (1 - z_0)^2 = (1 - \beta)^2 = \beta$. Par récurrence immédiate, $z_n = \beta$ pour tout $n \in \mathbb{N}$. La suite (z_n) est la suite constante $(\beta, \beta, \beta, \dots)$. L'ensemble des valeurs d'adhérence de (z_n) est $S_z = \{\beta\}$.

L'ensemble des valeurs d'adhérence de (x_n) est $S = S_y \cup S_z = \{\alpha, \beta\}$. Soit $S = \left\{\frac{\sqrt{5}-1}{2}, \frac{3-\sqrt{5}}{2}\right\}$.

c. Pour $x_0 = 0$, déterminer l'ensemble S . Justifier rigoureusement.

Soit $x_0 = 0$.

Pour la suite (y_n) : Nous avons $y_0 = x_0 = 0$. La relation de récurrence est $y_{n+1} = 1 - y_n^2$. Calculons les premiers termes de (y_n) : $y_0 = 0$, $y_1 = 1 - y_0^2 = 1 - 0^2 = 1$, $y_2 = 1 - y_1^2 = 1 - 1^2 = 0$, $y_3 = 1 - y_2^2 = 1 - 0^2 = 1$. La suite (y_n) est la suite périodique $(0, 1, 0, 1, \dots)$. Les valeurs d'adhérence de (y_n) sont les limites des sous-suites convergentes. La sous-suite des termes d'indices pairs (y_{2k}) est $(0, 0, 0, \dots)$, qui converge vers 0. La sous-suite des termes d'indices impairs (y_{2k+1}) est $(1, 1, 1, \dots)$, qui converge vers 1. Toute autre sous-suite de (y_n) doit contenir une infinité de 0 ou une infinité de 1 (ou les deux). Si elle converge, sa limite doit être 0 ou 1. Donc, l'ensemble des valeurs d'adhérence de (y_n) est $S_y = \{0, 1\}$.

Pour la suite (z_n) : Nous avons $z_0 = x_0^2 = 0^2 = 0$. La relation de récurrence est $z_{n+1} = (1 - z_n)^2$. Calculons les premiers termes de (z_n) : $z_0 = 0$, $z_1 = (1 - z_0)^2 = (1 - 0)^2 = 1^2 = 1$, $z_2 = (1 - z_1)^2 = (1 - 1)^2 = 0^2 = 0$, $z_3 = (1 - z_2)^2 = (1 - 0)^2 = 1^2 = 1$. La suite (z_n) est la suite périodique $(0, 1, 0, 1, \dots)$. De manière similaire à (y_n) , l'ensemble des valeurs d'adhérence de (z_n) est $S_z = \{0, 1\}$.

L'ensemble des valeurs d'adhérence de (x_n) est $S = S_y \cup S_z = \{0, 1\} \cup \{0, 1\} = \{0, 1\}$.

Conclusion : Pour $x_0 = 0$, l'ensemble des valeurs d'adhérence de la suite (x_n) est $S = \{0, 1\}$.

Correction de l'Exercice 10

Question 1 : Démontrer que l'ensemble V est un ensemble fermé de $\mathbb{R}^d$.

Pour démontrer que V est fermé, nous allons montrer que toute suite convergente de points de V a sa limite dans V . Soit $(L_j)_{j \in \mathbb{N}}$ une suite de points de V qui converge vers un point $L \in \mathbb{R}^d$. Nous devons montrer que $L \in V$.

Puisque chaque $L_j \in V$, par définition, pour chaque $j \in \mathbb{N}$, il existe une sous-suite $(x_{\varphi_j(k)})_{k \in \mathbb{N}}$ de (x_n) qui converge vers L_j . Nous allons construire une sous-suite de (x_n) qui converge vers L .

Pour chaque $j \in \mathbb{N}$, comme $L_j = \lim_{k \rightarrow \infty} x_{\varphi_j(k)}$, il existe un indice k_j tel que pour tout $k \geq k_j$, $\|x_{\varphi_j(k)} - L_j\| < \frac{1}{j+1}$. De plus, comme $L = \lim_{j \rightarrow \infty} L_j$, pour tout $\varepsilon > 0$, il existe un indice $J \in \mathbb{N}$ tel que pour tout $j \geq J$, $\|L_j - L\| < \varepsilon/2$.

Nous construisons la sous-suite $(x_{\psi(m)})_{m \in \mathbb{N}}$ de la manière suivante : Pour $m = 0$: Puisque $L_0 \in V$, il existe une sous-suite $(x_{\varphi_0(k)})$ convergeant vers L_0 . Nous pouvons choisir un indice $n_0 = \varphi_0(k_0)$ tel que $\|x_{n_0} - L_0\| < 1$. Nous posons $\psi(0) = n_0$. Pour $m = 1$: Puisque $L_1 \in V$, il existe une sous-suite $(x_{\varphi_1(k)})$ convergeant vers L_1 . Nous pouvons choisir un indice $n_1 = \varphi_1(k_1)$ tel que $n_1 > \psi(0)$ et $\|x_{n_1} - L_1\| < 1/2$. Nous posons $\psi(1) = n_1$. En général, supposons que $\psi(m-1)$ a été choisi. Puisque $L_m \in V$, il existe une sous-suite $(x_{\varphi_m(k)})$ convergeant vers L_m . Nous pouvons choisir un indice $n_m = \varphi_m(k_m)$ tel que $n_m > \psi(m-1)$ et $\|x_{n_m} - L_m\| < \frac{1}{m+1}$. Nous posons $\psi(m) = n_m$. Cette construction est toujours possible car chaque sous-suite $(x_{\varphi_m(k)})$ a une infinité de termes, ce qui nous permet de choisir un indice $\varphi_m(k_m)$ arbitrairement grand. La suite d'indices $(\psi(m))_{m \in \mathbb{N}}$ est strictement croissante par construction.

Maintenant, nous allons montrer que la sous-suite $(x_{\psi(m)})_{m \in \mathbb{N}}$ converge vers L . Pour tout $m \in \mathbb{N}$, nous avons : $\|x_{\psi(m)} - L\| = \|x_{\psi(m)} - L_m + L_m - L\|$. Par l'inégalité triangulaire, nous obtenons : $\|x_{\psi(m)} - L\| \leq \|x_{\psi(m)} - L_m\| + \|L_m - L\|$.

Par construction, nous avons $\|x_{\psi(m)} - L_m\| < \frac{1}{m+1}$. De plus, comme la suite (L_j) converge vers L , pour tout $\varepsilon > 0$, il existe un entier $J \in \mathbb{N}$ tel que pour tout $m \geq J$, $\|L_m - L\| < \varepsilon/2$. Choisissons $M_0 \in \mathbb{N}$ tel que pour tout $m \geq M_0$, $\frac{1}{m+1} < \varepsilon/2$. Alors, pour tout $m \geq \max(J, M_0)$, nous avons : $\|x_{\psi(m)} - L\| < \frac{1}{m+1} + \|L_m - L\| < \varepsilon/2 + \varepsilon/2 = \varepsilon$.

Ceci prouve que la sous-suite $(x_{\psi(m)})_{m \in \mathbb{N}}$ converge vers L . Par conséquent, L est une valeur d'adhérence de (x_n) , ce qui signifie $L \in V$. L'ensemble V est donc fermé.

Question 2 : Démontrer que si la suite (x_n) est bornée, alors V est un ensemble non vide et compact.

V est non vide : Puisque la suite (x_n) est bornée, il existe une constante $M > 0$ telle que pour tout $n \in \mathbb{N}$, $\|x_n\| \leq M$. Le théorème de Bolzano-Weierstrass dans $\mathbb{R}^d$ stipule que toute suite bornée dans $\mathbb{R}^d$ admet au moins une sous-suite convergente. Soit $(x_{\varphi(k)})_{k \in \mathbb{N}}$ une telle sous-suite convergente. Sa limite $L = \lim_{k \rightarrow \infty} x_{\varphi(k)}$ est, par définition, une valeur d'adhérence de (x_n) . Donc, $L \in V$, ce qui prouve que V est non vide.

V est compact : Dans $\mathbb{R}^d$, un ensemble est compact si et seulement s'il est fermé et borné (théorème de Heine-Borel). Nous avons déjà démontré à la question 1 que V est un ensemble fermé. Il nous reste à montrer que V est borné.

Puisque la suite (x_n) est bornée, il existe une constante $M > 0$ telle que pour tout $n \in \mathbb{N}$, $\|x_n\| \leq M$. Soit $L \in V$. Par définition, il existe une sous-suite $(x_{\varphi(k)})_{k \in \mathbb{N}}$ qui converge vers L . Puisque tous les termes de la suite (x_n) sont bornés par M , tous les termes de la sous-suite $(x_{\varphi(k)})$ sont également bornés par M , c'est-à-dire $\|x_{\varphi(k)}\| \leq M$ pour tout $k \in \mathbb{N}$. Par une propriété fondamentale des limites, si une suite (y_k) converge vers L et que $\|y_k\| \leq M$ pour tout k , alors $\|L\| \leq M$. Pour le prouver, supposons par l'absurde que $\|L\| > M$. Posons $\varepsilon = (\|L\| - M)/2$. Alors $\varepsilon > 0$. Puisque $(x_{\varphi(k)})$ converge vers L , il existe un $K \in \mathbb{N}$ tel que pour tout $k \geq K$, $\|x_{\varphi(k)} - L\| < \varepsilon$. En utilisant l'inégalité triangulaire inverse, nous avons $\|L\| - \|x_{\varphi(k)}\| \leq \|L - x_{\varphi(k)}\| < \varepsilon$. Donc, $\|L\| - \|x_{\varphi(k)}\| < \varepsilon$, ce qui implique $\|x_{\varphi(k)}\| > \|L\| - \varepsilon$. En substituant la valeur de ε : $\|x_{\varphi(k)}\| > \|L\| - (\|L\| - M)/2 = (\|L\| + M)/2$. Puisque $\|L\| > M$, on a $(\|L\| + M)/2 > (M + M)/2 = M$. Ainsi, pour

$k \geq K$, nous aurions $\|x_{\varphi(k)}\| > M$, ce qui contredit l'hypothèse que $\|x_n\| \leq M$ pour tout n . Par conséquent, notre supposition était fautive, et nous devons avoir $\|L\| \leq M$.

Puisque cette propriété est vraie pour tout $L \in V$, cela signifie que V est contenu dans la boule fermée $B(0, M)$, et donc V est borné. Puisque V est fermé et borné, il est compact d'après le théorème de Heine-Borel.

Question 3 : Démontrer que $V = \bigcap_{n \in \mathbb{N}} \overline{S_n}$.

Rappelons que $S_n = \{x_k \mid k \geq n\}$.

Partie 1 : Montrons que $V \subseteq \bigcap_{n \in \mathbb{N}} \overline{S_n}$. Soit $L \in V$. Par définition, il existe une sous-suite $(x_{\varphi(k)})_{k \in \mathbb{N}}$ qui converge vers L . Nous devons montrer que $L \in \overline{S_n}$ pour tout $n \in \mathbb{N}$. Fixons un entier $n \in \mathbb{N}$. Puisque $\varphi(k) \rightarrow \infty$ lorsque $k \rightarrow \infty$, il existe un entier K_n tel que pour tout $k \geq K_n$, $\varphi(k) \geq n$. Cela signifie que pour tout $k \geq K_n$, le terme $x_{\varphi(k)}$ appartient à l'ensemble $S_n = \{x_j \mid j \geq n\}$. La suite $(x_{\varphi(k)})_{k \geq K_n}$ est une suite de points de S_n qui converge vers L . Par définition de l'adhérence, un point L appartient à l'adhérence d'un ensemble A si et seulement s'il existe une suite de points de A qui converge vers L . Puisque L est la limite d'une suite de points de S_n , $L \in \overline{S_n}$. Cette conclusion est valable pour tout $n \in \mathbb{N}$. Par conséquent, $L \in \bigcap_{n \in \mathbb{N}} \overline{S_n}$. Ceci prouve que $V \subseteq \bigcap_{n \in \mathbb{N}} \overline{S_n}$.

Partie 2 : Montrons que $\bigcap_{n \in \mathbb{N}} \overline{S_n} \subseteq V$. Soit $L \in \bigcap_{n \in \mathbb{N}} \overline{S_n}$. Cela signifie que pour tout $n \in \mathbb{N}$, $L \in \overline{S_n}$. Nous allons construire une sous-suite $(x_{\psi(m)})_{m \in \mathbb{N}}$ qui converge vers L .

Pour $m = 0$: Puisque $L \in \overline{S_0}$, par définition de l'adhérence, il existe un point $y_0 \in S_0$ tel que $\|y_0 - L\| < 1$. Puisque $y_0 \in S_0$, $y_0 = x_k$ pour un certain $k \geq 0$. Nous posons $\psi(0) = k$. Pour $m = 1$: Puisque $L \in \overline{S_{\psi(0)+1}}$, il existe un point $y_1 \in S_{\psi(0)+1}$ tel que $\|y_1 - L\| < 1/2$. Puisque $y_1 \in S_{\psi(0)+1}$, $y_1 = x_k$ pour un certain $k \geq \psi(0) + 1$. Nous posons $\psi(1) = k$. Par construction, $\psi(1) > \psi(0)$. En général, supposons que $\psi(m-1)$ a été choisi. Puisque $L \in \overline{S_{\psi(m-1)+1}}$, il existe un point $y_m \in S_{\psi(m-1)+1}$ tel que $\|y_m - L\| < \frac{1}{m+1}$. Puisque $y_m \in S_{\psi(m-1)+1}$, $y_m = x_k$ pour un certain $k \geq \psi(m-1) + 1$. Nous posons $\psi(m) = k$. Par construction, la suite d'indices $(\psi(m))_{m \in \mathbb{N}}$ est strictement croissante, donc $(x_{\psi(m)})$ est bien une sous-suite de (x_n) . De plus, par construction, nous avons $\|x_{\psi(m)} - L\| < \frac{1}{m+1}$. Comme $\lim_{m \rightarrow \infty} \frac{1}{m+1} = 0$, il s'ensuit que $\lim_{m \rightarrow \infty} x_{\psi(m)} = L$. Donc, L est une valeur d'adhérence de la suite (x_n) , ce qui signifie $L \in V$. Ceci prouve que $\bigcap_{n \in \mathbb{N}} \overline{S_n} \subseteq V$.

En combinant les deux parties, nous avons démontré que $V = \bigcap_{n \in \mathbb{N}} \overline{S_n}$.

Question 4 : Suite $x_n = (\cos(n\theta), \sin(n\theta))$ dans $\mathbb{R}^2$.

La suite (x_n) est une suite de points sur le cercle unité $C = \{(x, y) \in \mathbb{R}^2 \mid x^2 + y^2 = 1\}$. Le cercle unité est un ensemble fermé et borné dans $\mathbb{R}^2$, donc il est compact d'après le théorème de Heine-Borel. Puisque tous les termes de la suite (x_n) sont sur le cercle unité, la suite est bornée. D'après la question 2, l'ensemble V des valeurs d'adhérence est non vide et compact. De plus, $V \subseteq C$.

a. Cas où $\theta = 2\pi p/q$ pour $p \in \mathbb{Z}$ et $q \in \mathbb{N}^*$.

Dans ce cas, θ est un multiple rationnel de 2π . Les termes de la suite sont $x_n = (\cos(n \cdot 2\pi p/q), \sin(n \cdot 2\pi p/q))$. Considérons le terme x_{n+q} : $x_{n+q} = (\cos((n+q) \cdot 2\pi p/q), \sin((n+q) \cdot 2\pi p/q))$ $x_{n+q} = (\cos(n \cdot 2\pi p/q + q \cdot 2\pi p/q), \sin(n \cdot 2\pi p/q + q \cdot 2\pi p/q))$ $x_{n+q} = (\cos(n \cdot 2\pi p/q + 2\pi p), \sin(n \cdot 2\pi p/q + 2\pi p))$ Puisque les fonctions cosinus et sinus sont 2π -périodiques, et $2\pi p$ est un multiple entier de 2π : $x_{n+q} = (\cos(n \cdot 2\pi p/q), \sin(n \cdot 2\pi p/q)) = x_n$. La suite (x_n) est donc périodique de période q . Par conséquent, la suite ne prend qu'un nombre fini de valeurs distinctes. Ces valeurs sont $\{x_0, x_1, \dots, x_{q-1}\}$. L'ensemble des valeurs d'adhérence V d'une suite qui ne prend qu'un nombre fini de valeurs est précisément l'ensemble de ces valeurs elles-mêmes. Pour le démontrer : 1. **Si $L \in V$** : Il existe une sous-suite $(x_{\varphi(k)})$ qui converge vers L . Puisque la suite (x_n) ne prend qu'un nombre fini de valeurs, la sous-suite $(x_{\varphi(k)})$ doit prendre au moins une de ces valeurs une infinité de fois. Soit x_j cette valeur. Alors la sous-suite $(x_{\varphi(k)})$ contient une sous-sous-suite constante égale à x_j . Cette sous-sous-suite converge vers x_j . Par unicité de la limite, $L = x_j$. Donc L est l'une des valeurs prises par la suite. 2. **Si x_j est une valeur prise par la suite** : Puisque la suite est périodique de période q , la valeur x_j apparaît une infinité de fois dans la suite (par exemple, $x_j, x_{j+q}, x_{j+2q}, \dots$). Cette sous-suite constante $(x_{j+kq})_{k \in \mathbb{N}}$ converge vers x_j . Donc x_j est une valeur d'adhérence.

Ainsi, l'ensemble des valeurs d'adhérence est $V = \{x_0, x_1, \dots, x_{q-1}\}$. Ces points sont $x_k = (\cos(k \cdot 2\pi p/q), \sin(k \cdot 2\pi p/q))$ pour $k \in \{0, 1, \dots, q-1\}$. Il est à noter que si p et q ne sont pas premiers entre eux, le nombre de points distincts peut être inférieur à q . Plus précisément, le nombre de points distincts est $q/\text{pgcd}(p, q)$.

b. Cas où $\theta/(2\pi)$ est un nombre irrationnel.

Dans ce cas, nous allons montrer que l'ensemble des valeurs d'adhérence V est le cercle unité tout entier, $C = \{(x, y) \in \mathbb{R}^2 \mid x^2 + y^2 = 1\}$. Pour cela, nous allons utiliser un résultat fondamental de la théorie des nombres : le théorème de Kronecker (ou un cas particulier de celui-ci). Le théorème de Kronecker affirme que si α est un nombre irrationnel, alors l'ensemble $\{n\alpha - \lfloor n\alpha \rfloor \mid n \in \mathbb{N}\}$ est dense dans $[0, 1]$. En d'autres termes, l'ensemble des parties fractionnaires de $n\alpha$ est dense dans $[0, 1]$. Si nous posons $\alpha = \theta/(2\pi)$, alors α est irrationnel par hypothèse. L'ensemble $\{n\theta/(2\pi) \pmod{1} \mid n \in \mathbb{N}\}$ est dense dans $[0, 1]$. Ceci est équivalent à dire que l'ensemble $\{n\theta \pmod{2\pi} \mid n \in \mathbb{N}\}$ est dense dans $[0, 2\pi]$.

Preuve de la densité de $\{n\theta \pmod{2\pi}\}$ dans $[0, 2\pi]$ (esquisse, car c'est un résultat classique) : Soit $y_n = n\theta \pmod{2\pi}$. 1. **Les points y_n sont distincts :** Si $y_n = y_m$ pour $n \neq m$, alors $n\theta - m\theta = k \cdot 2\pi$ pour un certain entier k . Cela implique $(n - m)\theta = k \cdot 2\pi$, et donc $\theta/(2\pi) = k/(n - m)$, ce qui est un nombre rationnel. Ceci contredit l'hypothèse que $\theta/(2\pi)$ est irrationnel. Donc tous les y_n sont distincts. 2. **Densité :** Puisqu'il y a une infinité de points distincts y_n dans l'intervalle borné $[0, 2\pi]$, il doit y avoir des points arbitrairement proches les uns des autres. Pour tout $N \in \mathbb{N}^*$, considérons les $N + 1$ points $y_0, y_1, \dots, y_N$ dans $[0, 2\pi]$. Divisons l'intervalle $[0, 2\pi]$ en N sous-intervalles de longueur $2\pi/N$. Par le principe des tiroirs de Dirichlet, il doit y avoir au moins deux points y_j, y_k (avec $0 \leq j < k \leq N$) qui tombent dans le même sous-intervalle. Donc, $|y_k - y_j| < 2\pi/N$. Soit $\delta = |y_k - y_j|$. Alors $\delta = |(k - j)\theta - l \cdot 2\pi|$ pour un certain entier l . Ainsi, δ est de la forme $m\theta \pmod{2\pi}$ pour $m = k - j \neq 0$. Les multiples de δ , c'est-à-dire $\{p\delta \pmod{2\pi} \mid p \in \mathbb{N}\}$, sont également de la forme $p(k - j)\theta \pmod{2\pi}$, donc ils sont des points de la suite $\{n\theta \pmod{2\pi}\}$. Puisque δ peut être rendu arbitrairement petit en choisissant N suffisamment grand, l'ensemble $\{p\delta \pmod{2\pi}\}$ est dense dans $[0, 2\pi]$. Par conséquent, l'ensemble $\{n\theta \pmod{2\pi} \mid n \in \mathbb{N}\}$ est dense dans $[0, 2\pi]$.

Maintenant, revenons à la suite $x_n = (\cos(n\theta), \sin(n\theta))$. L'ensemble des points x_n est l'image de l'ensemble $\{n\theta \pmod{2\pi} \mid n \in \mathbb{N}\}$ par l'application $f : t \mapsto (\cos t, \sin t)$. L'application $f : [0, 2\pi] \rightarrow C$ est continue et surjective. Puisque l'ensemble $\{n\theta \pmod{2\pi} \mid n \in \mathbb{N}\}$ est dense dans $[0, 2\pi]$, et que f est une application continue, l'image de cet ensemble, c'est-à-dire $\{x_n \mid n \in \mathbb{N}\}$, est dense dans l'image de $[0, 2\pi]$ par f . L'image de $[0, 2\pi]$ par f est le cercle unité C . Donc, l'ensemble $A = \{x_n \mid n \in \mathbb{N}\}$ est dense dans le cercle unité C .

L'ensemble des valeurs d'adhérence V d'une suite est égal à l'adhérence de l'ensemble des valeurs prises par la suite, c'est-à-dire $V = \overline{A}$. Puisque A est dense dans C , son adhérence est C . Donc, $V = C$.

En résumé, lorsque $\theta/(2\pi)$ est un nombre irrationnel, l'ensemble des valeurs d'adhérence de la suite $x_n = (\cos(n\theta), \sin(n\theta))$ est le cercle unité tout entier.

Travaux Pratiques

TP 1 - Jalon 15 : Bolzano-Weierstrass par Dichotomie

Objectif Mathématique

Le théorème de Bolzano-Weierstrass est un résultat fondamental en analyse réelle. Il stipule que **toute suite réelle bornée admet au moins une sous-suite convergente**. Ce théorème est d'une importance capitale car il garantit l'existence de points d'accumulation pour les suites bornées, même si la suite elle-même ne converge pas.

L'objectif de ce TP est de comprendre et d'implémenter algorithmiquement une preuve constructive de ce théorème en utilisant la méthode de dichotomie (ou bisection). Nous allons développer un programme Python qui, étant donnée une suite bornée, va extraire une sous-suite dont les termes se rapprochent de plus en plus, illustrant ainsi sa convergence.

Concepts clés à revoir :

- * **Suite bornée :** Une suite $(x_n)_{n \in \mathbb{N}}$ est bornée s'il existe un réel $M > 0$ tel que $|x_n| \leq M$ pour tout n .
- * **Sous-suite :** Une sous-suite de (x_n) est une suite $(x_{n_k})_{k \in \mathbb{N}}$ où (n_k) est une suite strictement croissante d'entiers naturels.
- * **Convergence d'une suite :** Une suite (y_k) converge vers L si pour tout $\epsilon > 0$, il existe un entier N tel que pour tout $k \geq N$, $|y_k - L| < \epsilon$.

L'algorithme de dichotomie permet de construire une suite d'intervalles emboîtés dont la longueur tend vers zéro, chacun contenant une infinité de termes de la suite originale (ou, dans le cas d'une suite finie, un nombre suffisant de termes restants). En choisissant un terme de la suite dans chaque intervalle successif, en s'assurant que les indices sont croissants, on construit la sous-suite convergente.

Code Python

Nous allons implémenter deux fonctions : 1. `generate_random_bounded_sequence` : Pour créer une suite bornée aléatoire à des fins de test. 2. `find_bolzano_weierstrass_subsequence` : L'implémentation principale de l'algorithme de dichotomie pour extraire la sous-suite.

Le code sera écrit en Python pur, sans l'utilisation de bibliothèques de haut niveau comme `numpy` ou `itertools` pour la logique algorithmique. Le module `random` est utilisé uniquement pour la génération de données de test et est considéré comme une bibliothèque standard de base.

```
import random

def generate_random_bounded_sequence(length: int, lower_bound: float, upper_bound: float) -> list[float]:
    """
    Génère une suite aléatoire de nombres flottants bornée.

    Args:
        length: La longueur de la suite à générer.
        lower_bound: La borne inférieure de l'intervalle des valeurs possibles.
        upper_bound: La borne supérieure de l'intervalle des valeurs possibles.

    Returns:
        Une liste de flottants représentant la suite bornée.
    """
    assert length > 0, "La longueur de la suite doit être positive."
    assert lower_bound <= upper_bound, "La borne inférieure doit être inférieure ou égale à la borne supérieure."

    sequence: list[float] = []
```

```

for _ in range(length):
    sequence.append(random.uniform(lower_bound, upper_bound))
return sequence

def find_bolzano_weierstrass_subsequence(
    sequence: list[float],
    max_subsequence_length: int
) -> list[float]:
    """
    Extrait une sous-suite convergente d'une suite bornée en utilisant
    une approche inspirée de l'algorithme de dichotomie de Bolzano-Weierstrass.

    L'algorithme procède par bisection d'intervalles. À chaque étape, il choisit
    la moitié de l'intervalle qui contient des termes de la suite avec des indices
    strictement supérieurs à celui du dernier terme choisi. Cela garantit que
    les indices de la sous-suite sont croissants et que les termes se resserrent
    dans un intervalle de plus en plus petit, assurant la convergence.

    Args:
        sequence: La suite bornée d'où extraire la sous-suite.
        max_subsequence_length: La longueur maximale souhaitée pour la sous-suite.
                                La sous-suite peut être plus courte si la suite
                                originale
                                est épuisée ou si aucune autre terme ne peut être
                                trouvé
                                avec un indice croissant.

    Returns:
        Une liste de flottants représentant la sous-suite extraite qui est
        convergente.
    """
    assert len(sequence) > 0, "La suite ne doit pas être vide."
    assert max_subsequence_length > 0, "La longueur maximale de la sous-suite doit être positive."

    # 1. Trouver les bornes initiales de la suite
    # Ces bornes définissent l'intervalle initial [min_val, max_val] qui contient
    # tous les termes.
    min_val: float = sequence[0]
    max_val: float = sequence[0]
    for x in sequence:
        if x < min_val:
            min_val = x
        if x > max_val:
            max_val = x

    current_lower_bound: float = min_val
    current_upper_bound: float = max_val

    subsequence: list[float] = []
    # last_chosen_index garde en mémoire l'indice du dernier élément choisi dans la
    # suite originale.
    # Il est initialisé à -1 pour permettre de choisir le premier élément de la suite
    # .
    last_chosen_index: int = -1

    # 2. Boucle pour construire la sous-suite
    # On itère pour trouver jusqu'à 'max_subsequence_length' éléments.
    for _ in range(max_subsequence_length):
        # Calcul du point milieu de l'intervalle courant
        mid: float = (current_lower_bound + current_upper_bound) / 2.0

        # Chercher des candidats dans la moitié gauche et droite de l'intervalle
        # en s'assurant que leur indice est strictement supérieur à '
        #     last_chosen_index'.
        left_candidates_indices: list[int] = []

```

```

right_candidates_indices: list[int] = []

for i in range(last_chosen_index + 1, len(sequence)):
    if sequence[i] <= mid:
        left_candidates_indices.append(i)
    else: # sequence[i] > mid
        right_candidates_indices.append(i)

chosen_candidates_indices: list[int] = []

# Choisir la moitié de l'intervalle qui contient des candidats.
# La priorité est donnée à la moitié gauche si les deux en ont,
# pour assurer un comportement déterministe.
if len(left_candidates_indices) > 0:
    chosen_candidates_indices = left_candidates_indices
    current_upper_bound = mid # Réduire l'intervalle à la moitié gauche
elif len(right_candidates_indices) > 0:
    chosen_candidates_indices = right_candidates_indices
    current_lower_bound = mid # Réduire l'intervalle à la moitié droite
else:
    # Si aucune des moitiés ne contient de termes avec un indice supérieur
    # à 'last_chosen_index', cela signifie que nous avons épuisé les termes
    # pertinents de la suite originale. La construction de la sous-suite s'
    # arrête.
    break

# Choisir le premier (plus petit indice) candidat disponible dans la moitié
# choisie.
# Cela garantit que les indices de la sous-suite sont strictement croissants.
n_k: int = chosen_candidates_indices[0] # chosen_candidates_indices est
    garanti non vide ici

subsequence.append(sequence[n_k])
last_chosen_index = n_k

return subsequence

# --- Exemple d'utilisation ---
if __name__ == "__main__":
    print("--- Démonstration du Théorème de Bolzano-Weierstrass ---")

    # 1. Générer une suite aléatoire bornée
    sequence_length: int = 200
    lower_bound: float = -5.0
    upper_bound: float = 5.0
    random_sequence: list[float] = generate_random_bounded_sequence(sequence_length,
        lower_bound, upper_bound)

    print(f"\nSuite originale (extrait des 10 premiers termes): {random_sequence
        [:10]}...")
    print(f"Longueur de la suite originale: {len(random_sequence)}")
    print(f"Bornes de la suite originale: [{min(random_sequence):.2f}, {max(
        random_sequence):.2f}]")

    # 2. Extraire une sous-suite convergente
    max_sub_len: int = 30 # Longueur maximale souhaitée pour la sous-suite
    convergent_subsequence: list[float] = find_bolzano_weierstrass_subsequence(
        random_sequence, max_sub_len)

    print(f"\nSous-suite convergente extraite (longueur {len(convergent_subsequence)
        }):")
    for i, val in enumerate(convergent_subsequence):
        print(f"  x_{i} = {val:.6f}")

    # 3. Vérification empirique de la convergence
    # En observant les différences entre termes consécutifs, on peut voir s'ils se

```

```

    rapprochent.
if len(convergent_subsequence) > 1:
    print("\nVérification empirique de la 'convergence' (différences absolues
          entre termes consécutifs):")
    for i in range(len(convergent_subsequence) - 1):
        diff = abs(convergent_subsequence[i+1] - convergent_subsequence[i])
        print(f" |x_{i+1} - x_{i}| = {diff:.6f}")
    print(f"Le dernier terme de la sous-suite est: {convergent_subsequence[-1]:.6f}")
    print("Les différences devraient tendre vers zéro, indiquant une convergence
          vers une valeur.")

# --- Test avec une suite plus structurée (mais toujours bornée) ---
print("\n--- Test avec une suite simple et bornée (ex: une suite oscillante) ---")
)
simple_sequence_length: int = 150
# Créons une suite qui oscille entre -1 et 1, par exemple (n % 100) / 50.0 - 1.0
# Cette suite est bornée et n'est pas convergente elle-même.
simple_sequence: list[float] = [(float(n % 100) / 50.0 - 1.0) for n in range(
    simple_sequence_length)]

print(f"\nSuite simple (extrait des 10 premiers termes): {simple_sequence
    [:10]}...")
print(f"Longueur de la suite simple: {len(simple_sequence)}")
print(f"Bornes de la suite simple: [{min(simple_sequence):.2f}, {max(
    simple_sequence):.2f}]")

max_sub_len_simple: int = 30
convergent_subsequence_simple: list[float] = find_bolzano_weierstrass_subsequence
    (simple_sequence, max_sub_len_simple)

print(f"\nSous-suite convergente extraite de la suite simple (longueur {len(
    convergent_subsequence_simple}):")
for i, val in enumerate(convergent_subsequence_simple):
    print(f" x_{i} = {val:.6f}")

if len(convergent_subsequence_simple) > 1:
    print("\nVérification empirique de la 'convergence' pour la suite simple:")
    for i in range(len(convergent_subsequence_simple) - 1):
        diff = abs(convergent_subsequence_simple[i+1] -
            convergent_subsequence_simple[i])
        print(f" |x_{i+1} - x_{i}| = {diff:.6f}")
    print(f"Le dernier terme de la sous-suite est: {convergent_subsequence_simple
        [-1]:.6f}")
    print("On observe que les termes de la sous-suite se rapprochent d'une valeur
        limite.")

```

Explications

Le Théorème de Bolzano-Weierstrass et la Dichotomie

Le cœur de ce TP est l'application constructive du théorème de Bolzano-Weierstrass. Pour une suite bornée (x_n) , nous savons qu'elle est contenue dans un intervalle fermé et borné $[a, b]$. L'idée de la preuve par dichotomie est la suivante :

1. **Initialisation** : On commence avec un intervalle $[a_0, b_0]$ qui contient tous les termes de la suite (x_n) .
2. **Bisection** : On divise cet intervalle en deux sous-intervalles de même longueur : $[a_0, m_0]$ et $[m_0, b_0]$, où $m_0 = (a_0 + b_0)/2$.
3. **Choix de l'intervalle** : Au moins l'un de ces deux sous-intervalles doit contenir une infinité de termes de la suite (x_n) . On choisit cet intervalle comme notre nouvel intervalle $[a_1, b_1]$. Si les deux en contiennent une infinité, on peut choisir l'un d'eux (par exemple, celui de gauche) pour rendre le processus déterministe.
4. **Extraction d'un terme** : On choisit un terme x_{n_1} de la suite originale qui se trouve dans $[a_1, b_1]$ et dont l'indice n_1 est strictement supérieur à l'indice du terme précédemment choisi (s'il y en a un).
5. **Itération** : On répète les étapes 2 à 4. À chaque étape k , on obtient un intervalle $[a_k, b_k]$ de longueur $(b_0 - a_0)/2^k$ et un terme x_{n_k} tel que $x_{n_k} \in [a_k, b_k]$ et $n_k > n_{k-1}$.

La suite d'intervalles emboîtés $([a_k, b_k])$ dont la longueur tend vers zéro garantit, par le théorème des intervalles emboîtés, qu'il existe un unique point L commun à tous ces intervalles. La sous-suite (x_{n_k}) ainsi construite converge vers ce point L .

Détails de l'Implémentation Python

Notre fonction `find_bolzano_weierstrass_subsequence` adapte cette preuve constructive pour une suite finie (`list[float]`) :

* **Bornes initiales** : La fonction commence par trouver les valeurs minimale et maximale de la suite d'entrée pour définir l'intervalle initial `[current_lower_bound, current_upper_bound]`. * **last_chosen_index** : Cette variable est cruciale. Elle assure que les indices des termes de la sous-suite (`n_k`) sont strictement croissants. À chaque itération, nous ne considérons que les termes de la suite originale dont l'indice est supérieur à `last_chosen_index`. **Dichotomie et sélection des candidats** : Dans la boucle principale, l'intervalle courant est divisé en deux par `mid`. Nous identifions ensuite les indices des termes de la suite restante* (ceux après `last_chosen_index`) qui tombent dans la moitié gauche (`left_candidates_indices`) et ceux qui tombent dans la moitié droite (`right_candidates_indices`). * **Choix de la moitié** : Pour une suite finie, l'idée d'une "infinité de termes" est remplacée par la présence de "termes disponibles". Nous choisissons la moitié qui contient au moins un candidat. Si les deux moitiés contiennent des candidats, nous privilégions la moitié gauche pour la reproductibilité. * **Extraction du terme** : Une fois la moitié choisie, nous prenons le terme avec le plus petit indice parmi les `chosen_candidates_indices`. Cela garantit la stricte croissance des indices de la sous-suite. Ce terme est ajouté à `subsequence`, et `last_chosen_index` est mis à jour. * **Condition d'arrêt** : La boucle s'exécute jusqu'à `max_subsequence_length` termes ou s'arrête plus tôt si aucun autre terme ne peut être trouvé avec un indice croissant dans les intervalles restants.

Limitations et Considérations

* **Suite finie vs. infinie** : Le théorème de Bolzano-Weierstrass s'applique aux suites infinies. Notre implémentation travaille sur une liste Python finie. Par conséquent, la "sous-suite convergente" que nous extrayons est une approximation. Si la suite originale est très courte, la sous-suite extraite le sera aussi et la "convergence" sera moins évidente. * **max_subsequence_length** : Ce paramètre contrôle la longueur de la sous-suite. Une sous-suite plus longue permettra une meilleure observation de la convergence. * **Précision flottante** : Comme toujours avec les nombres flottants, des problèmes de précision peuvent survenir, bien qu'ils soient généralement négligeables pour ce type d'algorithme.

Ce TP démontre comment un concept mathématique abstrait peut être traduit en un algorithme concret, fournissant une intuition plus profonde sur la nature des suites bornées et de leurs sous-suites convergentes.

Objectif Mathématique

Ce TP a pour objectif d'explorer et d'implémenter de manière constructive le **Théorème de Bolzano-Weierstrass**. Ce théorème fondamental de l'analyse réelle stipule que **toute suite réelle bornée admet une sous-suite convergente**.

La preuve constructive de ce théorème repose souvent sur un algorithme de dichotomie (ou de bisection). L'idée est la suivante : 1. Si une suite $(x_n)_{n \in \mathbb{N}}$ est bornée, elle est contenue dans un intervalle fermé et borné $I_0 = [a_0, b_0]$. 2. On divise I_0 en deux sous-intervalles de même longueur : $[a_0, m_0]$ et $[m_0, b_0]$ où $m_0 = (a_0 + b_0)/2$. Au moins l'un de ces deux sous-intervalles doit contenir une infinité de termes de la suite (x_n) . On choisit un tel sous-intervalle et on le nomme $I_1 = [a_1, b_1]$. 3. On répète ce processus : on divise I_k en deux, et on choisit I_{k+1} comme le sous-intervalle contenant une infinité de termes de la suite. 4. On obtient ainsi une suite d'intervalles emboîtés $(I_k)_{k \in \mathbb{N}}$ dont la longueur $b_k - a_k$ tend vers zéro. Par le théorème des intervalles emboîtés, l'intersection de ces intervalles est un unique point L . 5. Pour construire la sous-suite convergente $(x_{n_k})_{k \in \mathbb{N}}$, on choisit $x_{n_0} \in I_0$, puis $x_{n_1} \in I_1$ avec $n_1 > n_0$, et ainsi de suite, $x_{n_k} \in I_k$ avec $n_k > n_{k-1}$. Cette sous-suite converge vers L .

L'implémentation Python que nous allons réaliser cherchera à simuler cette construction pour extraire une telle sous-suite à partir d'une suite bornée donnée, en respectant la contrainte "from scratch" sans bibliothèques haut niveau.

Code Python

```
from typing import List, Tuple

def bolzano_weierstrass_dichotomy(
    sequence: List[float],
    epsilon: float = 1e-6,
    max_iterations: int = 1000
) -> List[float]:
    """
    Implémente l'algorithme de dichotomie pour extraire une sous-suite convergente
    d'une suite bornée, illustrant le théorème de Bolzano-Weierstrass.

    L'algorithme construit une suite d'intervalles emboîtés et sélectionne un élément
    de la suite originale dans chaque intervalle, en s'assurant que les indices
    de la sous-suite sont strictement croissants.

    Args:
        sequence (List[float]): La suite réelle bornée d'où extraire la sous-suite.
                                Doit contenir au moins un élément.
        epsilon (float): La précision souhaitée pour la longueur de l'intervalle
                        final.
                        La sous-suite convergera vers un point dans cet intervalle.
        max_iterations (int): Nombre maximal d'itérations pour éviter les boucles
                            infinies.

    Returns:
        List[float]: La sous-suite extraite qui converge vers un point.

    Raises:
        ValueError: Si la suite est vide, ou si epsilon/max_iterations sont invalides
    """
    # --- Validation des entrées ---
    if not sequence:
        raise ValueError("La suite ne peut pas être vide.")
    if epsilon <= 0:
        raise ValueError("Epsilon doit être un nombre positif.")
    if max_iterations <= 0:
        raise ValueError("max_iterations doit être un entier positif.")

    # --- Trouver les bornes initiales de la suite ---
    # Implémentation manuelle de min/max pour respecter "from scratch"
    min_val: float = sequence[0]
    max_val: float = sequence[0]
```

```

for x in sequence:
    if x < min_val:
        min_val = x
    if x > max_val:
        max_val = x

# Si la suite est constante, elle est déjà convergente
if min_val == max_val:
    return [min_val]

# --- Initialisation pour l'algorithme de dichotomie ---
subsequence: List[float] = []

# current_available_indices contient les indices des éléments de la suite
# originale
# qui sont encore "pertinents" pour la construction de la sous-suite dans l'
# intervalle courant.
# Au début, tous les indices sont pertinents.
current_available_indices: List[int] = list(range(len(sequence)))

# L'intervalle courant [a, b]
a: float = min_val
b: float = max_val

iterations: int = 0
last_chosen_index: int = -1 # Pour s'assurer que n_k > n_{k-1}

# --- Boucle principale de dichotomie ---
while (b - a) > epsilon and iterations < max_iterations and len(
    current_available_indices) > 0:
    m: float = (a + b) / 2 # Point milieu de l'intervalle

    # 1. Sélectionner un élément pour la sous-suite
    # On cherche le premier élément de la suite originale (avec un indice >
    # last_chosen_index)
    # qui se trouve dans l'intervalle courant [a, b].
    chosen_idx: int = -1
    for idx in current_available_indices:
        if idx > last_chosen_index and sequence[idx] >= a and sequence[idx] <= b:
            chosen_idx = idx
            break

    if chosen_idx == -1:
        # Plus d'éléments disponibles dans l'intervalle courant avec un indice
        # croissant.
        # La sous-suite ne peut plus être étendue selon cette règle.
        break

    subsequence.append(sequence[chosen_idx])
    last_chosen_index = chosen_idx

    # 2. Raffiner l'intervalle pour la prochaine itération
    # On détermine quelle moitié de l'intervalle [a, b] contient le plus d'élé
    # ments
    # de la suite originale dont l'indice est supérieur à last_chosen_index.

    left_half_indices: List[int] = []
    right_half_indices: List[int] = []

    for idx in current_available_indices:
        if idx > last_chosen_index: # Ne considérer que les éléments après le
            dernier choisi
            val = sequence[idx]
            if val >= a and val <= m:
                left_half_indices.append(idx)
            elif val > m and val <= b:

```

```

        right_half_indices.append(idx)

    # Choisir la moitié qui contient le plus d'éléments "disponibles"
    if len(left_half_indices) >= len(right_half_indices) and len(
        left_half_indices) > 0:
        b = m
        current_available_indices = left_half_indices
    elif len(right_half_indices) > 0:
        a = m
        current_available_indices = right_half_indices
    else:
        # Aucune des moitiés ne contient d'éléments disponibles (après
        # last_chosen_index)
        break

    iterations += 1

    return subsequence

# --- Exemples d'utilisation et assertions de validation ---

# Exemple 1: Suite convergente (devrait retourner la suite elle-même ou une partie)
seq1 = [1.0 / n for n in range(1, 101)] # Suite (1/n)
sub_seq1 = bolzano_weierstrass_dichotomy(seq1, epsilon=0.01)
print(f"Suite (1/n): {seq1[:10]}...")
print(f"Sous-suite extraite (1/n): {sub_seq1[:10]}...")
assert len(sub_seq1) > 0
assert all(sub_seq1[i] >= sub_seq1[i+1] for i in range(len(sub_seq1)-1)) # Devrait être décroissante
assert abs(sub_seq1[-1]) < 0.01 # Le dernier élément doit être proche de 0

# Exemple 2: Suite alternée bornée (cos(n) n'est pas simple, prenons une suite plus simple)
# Une suite qui alterne entre 0 et 1, bornée.
seq2 = [0.0 if i % 2 == 0 else 1.0 for i in range(20)]
sub_seq2 = bolzano_weierstrass_dichotomy(seq2, epsilon=0.1)
print(f"\nSuite alternée: {seq2}")
print(f"Sous-suite extraite alternée: {sub_seq2}")
# La sous-suite devrait converger vers 0 ou 1.
# Avec notre logique de "plus d'éléments", elle devrait converger vers la valeur la plus fréquente
# ou vers l'une des deux si elles sont également fréquentes.
# Ici, les deux sont également fréquentes, le choix dépendra de l'ordre et du point milieu.
# Si m=0.5, les 0.0 iront à gauche, les 1.0 à droite.
# Si len(left) == len(right), on choisit left. Donc on devrait converger vers 0.0.
assert len(sub_seq2) > 0
assert all(abs(x - 0.0) < 0.1 for x in sub_seq2) or all(abs(x - 1.0) < 0.1 for x in sub_seq2)
# Vérifions plus précisément:
# Initial [0,1], m=0.5. Left (0.0) and Right (1.0) have equal counts. We choose left.
# So it should converge to 0.0.
assert abs(sub_seq2[-1] - 0.0) < 0.1

# Exemple 3: Suite aléatoire bornée
import random
random.seed(42) # Pour la reproductibilité
seq3 = [random.uniform(-5.0, 5.0) for _ in range(100)]
sub_seq3 = bolzano_weierstrass_dichotomy(seq3, epsilon=0.5, max_iterations=50)
print(f"\nSuite aléatoire bornée: {seq3[:10]}...")
print(f"Sous-suite extraite aléatoire: {sub_seq3[:10]}...")
assert len(sub_seq3) > 0
# Vérifier que les éléments de la sous-suite sont dans l'intervalle final
final_a, final_b = -5.0, 5.0 # Re-calculer les bornes pour la vérification
current_a, current_b = final_a, final_b
for _ in range(50): # Simuler les 50 itérations pour trouver l'intervalle final

```

```

    m = (current_a + current_b) / 2
    count_left = sum(1 for x in seq3 if x >= current_a and x <= m)
    count_right = sum(1 for x in seq3 if x > m and x <= current_b)
    if count_left >= count_right and count_left > 0:
        current_b = m
    elif count_right > 0:
        current_a = m
    else:
        break
# Le dernier élément de la sous-suite doit être dans l'intervalle final
if sub_seq3:
    assert current_a <= sub_seq3[-1] <= current_b + epsilon # Ajout d'epsilon pour la
        tolérance
    assert (current_b - current_a) <= 0.5 + 1e-9 # L'intervalle final doit être <=
        epsilon

# Exemple 4: Cas d'une suite vide (doit lever une erreur)
try:
    bolzano_weierstrass_dichotomy([])
except ValueError as e:
    print(f"\nTest suite vide: {e}")
    assert "vide" in str(e)

# Exemple 5: Epsilon invalide
try:
    bolzano_weierstrass_dichotomy([1, 2, 3], epsilon=0)
except ValueError as e:
    print(f"Test epsilon invalide: {e}")
    assert "positif" in str(e)

# Exemple 6: max_iterations invalide
try:
    bolzano_weierstrass_dichotomy([1, 2, 3], max_iterations=0)
except ValueError as e:
    print(f"Test max_iterations invalide: {e}")
    assert "positif" in str(e)

# Exemple 7: Suite avec un seul élément
seq7 = [42.0]
sub_seq7 = bolzano_weierstrass_dichotomy(seq7)
print(f"\nSuite à un élément: {seq7}")
print(f"Sous-suite extraite: {sub_seq7}")
assert sub_seq7 == [42.0]

# Exemple 8: Suite où tous les éléments sont égaux
seq8 = [3.14, 3.14, 3.14, 3.14]
sub_seq8 = bolzano_weierstrass_dichotomy(seq8)
print(f"\nSuite constante: {seq8}")
print(f"Sous-suite extraite: {sub_seq8}")
assert sub_seq8 == [3.14]

```

Explications

L'implémentation ci-dessus suit fidèlement la logique de la preuve constructive du théorème de Bolzano-Weierstrass par dichotomie :

1. **Validation des entrées** : Avant tout traitement, la fonction vérifie la validité des arguments `sequence`, `epsilon` et `max_iterations` pour garantir un comportement correct et éviter des erreurs inattendues.

2. **Détermination des bornes initiales** : La première étape consiste à trouver les valeurs minimale et maximale de la suite `sequence`. Cela définit l'intervalle initial $[a, b]$ qui contient tous les termes de la suite. Si la suite est constante ($\text{min_val} == \text{max_val}$), elle est trivialement convergente, et la fonction retourne simplement cette valeur.

3. **Initialisation de la sous-suite et des indices** : * `subsequence` : Une liste vide qui sera remplie avec les éléments de la sous-suite extraite. *`current_available_indices` : Une liste d'indices de la suite originale. Au début, elle contient tous les indices. À chaque étape de dichotomie, cette liste est mise à jour pour ne*

contenir que les indices des éléments qui se trouvent dans le nouvel intervalle choisi et qui sont postérieurs* au dernier élément sélectionné pour la sous-suite. * `last_chosen_index` : Un entier qui garde en mémoire l'indice du dernier élément ajouté à `subsequence`. C'est crucial pour garantir que les indices de la sous-suite sont strictement croissants ($n_k > n_{k-1}$), une propriété fondamentale d'une sous-suite.

4. Boucle de Dichotomie Principale : * La boucle continue tant que la longueur de l'intervalle (`b - a`) est supérieure à `epsilon` (la précision désirée), que le nombre d'itérations n'a pas dépassé `max_iterations` (une sécurité), et qu'il reste des indices `current_available_indices` à considérer. * **Calcul du point milieu `m` :** L'intervalle `[a, b]` est divisé en deux moitiés : `[a, m]` et `(m, b]`. **Sélection d'un élément pour la sous-suite :** On parcourt `current_available_indices` pour trouver le premier* indice `chosen_idx` tel que `sequence[chosen_idx]` se trouve dans l'intervalle `[a, b]` et `chosen_idx` est strictement supérieur à `last_chosen_index`. Cet élément est ajouté à `subsequence`, et `last_chosen_index` est mis à jour. Cette étape garantit la construction d'une sous-suite avec des indices croissants. **Raffinement de l'intervalle :** Pour la prochaine itération, nous devons choisir l'une des deux moitiés (`[a, m]` ou `(m, b]`). La stratégie adoptée ici est de choisir la moitié qui contient le plus grand nombre d'éléments de la suite originale dont l'indice est strictement supérieur* à `last_chosen_index`. Cela simule l'idée de choisir l'intervalle qui contient "une infinité" de termes, en pratique, celui qui en contient le plus parmi ceux qui sont encore "disponibles". L'intervalle `[a, b]` et la liste `current_available_indices` sont mis à jour en conséquence. * Si aucune des moitiés ne contient d'éléments disponibles (après `last_chosen_index`), la boucle s'arrête car il n'est plus possible de construire la sous-suite selon les critères.

5. Retour de la sous-suite : Une fois la boucle terminée, la fonction retourne la `subsequence` construite. Cette sous-suite est garantie d'être convergente vers un point situé dans le dernier intervalle `[a, b]` dont la longueur est inférieure ou égale à `epsilon`.

Cette implémentation, bien que manuelle et sans l'aide de fonctions Python de haut niveau comme `min()`, `max()`, ou des modules comme `itertools`, démontre efficacement le principe de Bolzano-Weierstrass et la construction d'une sous-suite convergente par dichotomie.

Objectif Mathématique

Le **Théorème de Bolzano-Weierstrass** est un pilier de l'analyse réelle. Il stipule que toute suite réelle bornée admet une sous-suite convergente. En d'autres termes, si une suite $(x_n)_{n \in \mathbb{N}}$ est contenue dans un intervalle fermé $[a, b]$, alors il existe au moins un point $L \in [a, b]$ tel que l'on peut extraire de (x_n) une sous-suite (x_{n_k}) qui converge vers L . Ce point L est appelé un **point d'accumulation** ou **valeur d'adhérence** de la suite.

L'objectif de ce TP est d'implémenter un algorithme basé sur la **dichotomie** (ou méthode de bisection) pour trouver un tel point d'accumulation pour une suite bornée donnée. L'idée est de diviser récursivement l'intervalle de confinement de la suite en deux, en choisissant à chaque étape la moitié qui contient "le plus de termes" de la suite (ou, plus rigoureusement pour une suite infinie, une infinité de termes). En répétant ce processus, nous construisons une suite d'intervalles emboîtés dont la longueur tend vers zéro, et dont l'intersection est précisément un point d'accumulation.

Pour une implémentation pratique avec une suite finie (comme c'est le cas en programmation), nous ne pouvons pas littéralement trouver une "infinité de termes". Nous allons plutôt chercher l'intervalle qui contient la plus grande densité de points de la suite, ce qui nous mènera vers un point de forte concentration, simulant ainsi la recherche d'un point d'accumulation.

Code Python

Nous allons implémenter la fonction `find_cluster_point_bolzano_weierstrass` qui prend en entrée une suite de nombres flottants, ses bornes inférieure et supérieure, et un nombre maximal d'itérations pour la dichotomie. Elle retournera une estimation du point d'accumulation.

```
import random

def find_cluster_point_bolzano_weierstrass(
    sequence: list[float],
    lower_bound: float,
    upper_bound: float,
    max_iterations: int = 100
) -> float:
    """
    Implémente l'algorithme de dichotomie pour trouver un point d'accumulation
    (ou un point de forte concentration) d'une suite bornée, illustrant le
    principe du théorème de Bolzano-Weierstrass.

    Args:
        sequence: La suite de nombres flottants bornée.
        lower_bound: La borne inférieure de l'intervalle contenant la suite.
        upper_bound: La borne supérieure de l'intervalle contenant la suite.
        max_iterations: Le nombre d'itérations de dichotomie pour affiner
            la recherche du point. Plus ce nombre est élevé,
            plus la précision est grande.

    Returns:
        Une estimation du point d'accumulation de la suite.

    Raises:
        ValueError: Si les arguments ne respectent pas les contraintes.
    """
    # --- Assertions de validation des entrées ---
    if not isinstance(sequence, list):
        raise ValueError("La 'sequence' doit être une liste.")
    if not all(isinstance(x, (int, float)) for x in sequence):
        raise ValueError("Tous les éléments de la 'sequence' doivent être des nombres
            (int ou float).")
    if not sequence:
        raise ValueError("La 'sequence' ne doit pas être vide.")
    if not isinstance(lower_bound, (int, float)) or not isinstance(upper_bound, (int,
        float)):
        raise ValueError("'lower_bound' et 'upper_bound' doivent être des nombres.")
    if lower_bound >= upper_bound:
        raise ValueError("'lower_bound' doit être strictement inférieur à '
            upper_bound'.")
```

```

if not all(lower_bound <= x <= upper_bound for x in sequence):
    raise ValueError("Tous les éléments de la 'sequence' doivent être compris
        entre 'lower_bound' et 'upper_bound'.")
if not isinstance(max_iterations, int) or max_iterations <= 0:
    raise ValueError("'max_iterations' doit être un entier positif.")

# --- Initialisation de l'intervalle de recherche ---
current_lower: float = float(lower_bound)
current_upper: float = float(upper_bound)

# --- Boucle de dichotomie ---
for _ in range(max_iterations):
    mid: float = (current_lower + current_upper) / 2.0

    # Compter les éléments de la suite dans chaque sous-intervalle
    count_left: int = 0
    count_right: int = 0

    for x in sequence:
        if x <= mid:
            count_left += 1
        if x >= mid: # Note: mid peut être compté dans les deux, ce qui est
            # acceptable pour cette heuristique
            count_right += 1

    # Choisir le sous-intervalle qui contient le plus de points
    # (simule la recherche de l'intervalle contenant "une infinité" de points)
    if count_left >= count_right:
        current_upper = mid
    else:
        current_lower = mid

    # Le point d'accumulation est estimé par le milieu du dernier intervalle
    return (current_lower + current_upper) / 2.0

# --- Exemples d'utilisation ---

# Exemple 1: Suite simple avec un point d'accumulation évident
print("--- Exemple 1: Suite simple ---")
sequence_1: list[float] = [0.0, 1.0, 0.5, 0.25, 0.75, 0.125, 0.875, 0.0625, 0.9375]
# Cette suite a des points qui s'accumulent autour de 0 et 1.
# L'algorithme trouvera l'un des deux, ou un point intermédiaire si les densités sont
# égales.
# Pour une suite finie, il trouvera le point de plus forte concentration.
# Ici, les points sont plus ou moins équitablement répartis.
# Testons avec une borne inférieure et supérieure claires.
cluster_point_1: float = find_cluster_point_bolzano_weierstrass(sequence_1, 0.0, 1.0,
    max_iterations=20)
print(f"Suite 1: {sequence_1}")
print(f"Point d'accumulation estimé (Ex 1): {cluster_point_1:.5f}")
print("-" * 30)

# Exemple 2: Suite aléatoire avec un cluster artificiel
print("--- Exemple 2: Suite aléatoire avec cluster ---")
random.seed(42) # Pour la reproductibilité
sequence_2: list[float] = [random.uniform(0, 1) for _ in range(500)]
# Ajoutons un cluster de points autour de 0.7
sequence_2.extend([0.7 + random.uniform(-0.05, 0.05) for _ in range(300)])
# Ajoutons un autre cluster autour de 0.2
sequence_2.extend([0.2 + random.uniform(-0.03, 0.03) for _ in range(200)])
random.shuffle(sequence_2) # Mélanger pour ne pas avoir d'ordre particulier

cluster_point_2: float = find_cluster_point_bolzano_weierstrass(sequence_2, 0.0, 1.0,
    max_iterations=30)
print(f"Taille de la Suite 2: {len(sequence_2)}")
print(f"Point d'accumulation estimé (Ex 2): {cluster_point_2:.5f}")

```



```

# On s'attend à ce que le point trouvé soit proche de 0.7 car c'est le cluster le
# plus dense.
print("-" * 30)

# Exemple 3: Suite constante (le point d'accumulation est la valeur elle-même)
print("--- Exemple 3: Suite constante ---")
sequence_3: list[float] = [3.14 for _ in range(50)]
cluster_point_3: float = find_cluster_point_bolzano_weierstrass(sequence_3, 3.0, 3.5,
    max_iterations=15)
print(f"Suite 3: {sequence_3[:5]}...")
print(f"Point d'accumulation estimé (Ex 3): {cluster_point_3:.5f}")
print("-" * 30)

# Exemple 4: Test avec des bornes différentes
print("--- Exemple 4: Bornes différentes ---")
sequence_4: list[float] = [random.uniform(-10, 10) for _ in range(1000)]
sequence_4.extend([-5.0 + random.uniform(-0.1, 0.1) for _ in range(400)])
random.shuffle(sequence_4)
cluster_point_4: float = find_cluster_point_bolzano_weierstrass(sequence_4, -10.0,
    10.0, max_iterations=25)
print(f"Taille de la Suite 4: {len(sequence_4)}")
print(f"Point d'accumulation estimé (Ex 4): {cluster_point_4:.5f}")
print("-" * 30)

# Exemple 5: Test d'erreurs (décommenter pour tester)
# try:
#     find_cluster_point_bolzano_weierstrass([], 0.0, 1.0)
# except ValueError as e:
#     print(f"Erreur attendue: {e}")

# try:
#     find_cluster_point_bolzano_weierstrass([0.5], 1.0, 0.0)
# except ValueError as e:
#     print(f"Erreur attendue: {e}")

# try:
#     find_cluster_point_bolzano_weierstrass([0.5], 0.0, 0.2) # 0.5 n'est pas dans
#     [0, 0.2]
# except ValueError as e:
#     print(f"Erreur attendue: {e}")

```

Explications

1. **Théorème de Bolzano-Weierstrass et Dichotomie** : * Le théorème garantit l'existence d'un point d'accumulation pour toute suite bornée. L'algorithme de dichotomie est une méthode constructive pour trouver un tel point. * L'idée est de partir d'un intervalle initial $[a, b]$ qui contient tous les termes de la suite. * À chaque étape, nous divisons cet intervalle en deux sous-intervalles de même longueur : $[a, m]$ et $[m, b]$, où $m = (a+b)/2$. * Pour une suite infinie, le théorème nous dit qu'au moins un de ces sous-intervalles doit contenir une infinité de termes de la suite. Nous choisissons cet intervalle et répétons le processus. * Pour une suite finie (comme dans notre implémentation), nous ne pouvons pas parler d'infinité de termes. Nous utilisons une heuristique : nous choisissons le sous-intervalle qui contient le plus grand nombre de termes de la suite. Cela nous guide vers les régions où les points de la suite sont les plus "denses".

2. **Implémentation from scratch** : * Le code n'utilise aucune bibliothèque Python de haut niveau pour la logique algorithmique (pas de `numpy`, `scipy`, `itertools`, etc.). Toutes les opérations (calcul du milieu, comptage des éléments) sont réalisées avec des boucles et des opérations arithmétiques de base. * Le typage strict (`list[float]`, `float`, `int`) est utilisé pour améliorer la lisibilité et la robustesse du code, en spécifiant clairement les types attendus pour les arguments et le retour de la fonction.

3. **Assertions de Validation** : * Avant d'exécuter l'algorithme principal, des assertions (`if not ... raise ValueError`) sont utilisées pour valider les entrées. Cela garantit que la fonction reçoit des arguments valides (par exemple, la suite n'est pas vide, les bornes sont correctes, tous les éléments de la suite sont bien dans l'intervalle spécifié). C'est une bonne pratique de développement pour créer des fonctions robustes.

4. **Processus de Dichotomie** : * La boucle `for _ in range(max_iterations)` exécute la dichotomie un nombre fixe de fois. Chaque itération réduit la taille de l'intervalle de recherche de moitié. * Après `max_iterations`

itérations, la longueur de l'intervalle final est $(\text{upper_bound} - \text{lower_bound}) / (2 ** \text{max_iterations})$. Plus `max_iterations` est grand, plus l'intervalle final est petit, et plus l'estimation du point d'accumulation est précise. * Le point d'accumulation est finalement estimé par le milieu du dernier intervalle `[current_lower, current_upper]`.

5. Limitations et Interprétation : *Pour une suite finie, l'algorithme ne trouve pas une "sous-suite convergente" au sens strict d'une suite d'indices strictement croissante. Il trouve plutôt un point de concentration ou un point d'accumulation** de l'ensemble des valeurs de la suite. C'est le point autour duquel les valeurs de la suite sont les plus "serrées". * Si la suite a plusieurs points d'accumulation (par exemple, `[0, 1, 0, 1, 0, 1, ...]`), l'algorithme trouvera celui qui est "le plus dense" ou celui vers lequel la dichotomie converge en fonction des règles de décision (`count_left >= count_right`). * La précision du résultat dépend directement de `max_iterations`.

Ce TP démontre comment un concept mathématique abstrait comme le théorème de Bolzano-Weierstrass peut être concrétisé par un algorithme simple et efficace, en utilisant uniquement des outils de programmation de base.

Objectif Mathématique

Ce Travail Pratique explore les concepts fondamentaux de sous-suites et de valeurs d'adhérence, culminant avec une approche algorithmique pour "découvrir" empiriquement ces valeurs pour une suite donnée.

Une **sous-suite** d'une suite $(u_n)_{n \in \mathbb{N}}$ est une suite $(u_{\phi(k)})_{k \in \mathbb{N}}$ où $\phi : \mathbb{N} \rightarrow \mathbb{N}$ est une fonction strictement croissante. En d'autres termes, on sélectionne une infinité de termes de la suite originale en respectant leur ordre.

Une **valeur d'adhérence** (ou point d'accumulation) d'une suite $(u_n)_{n \in \mathbb{N}}$ est un réel L tel que toute boule ouverte centrée en L (c'est-à-dire tout intervalle $]L - \epsilon, L + \epsilon[$ pour $\epsilon > 0$) contient une infinité de termes de la suite. De manière équivalente, L est une valeur d'adhérence s'il existe une sous-suite de (u_n) qui converge vers L .

Le **Théorème de Bolzano-Weierstrass** est un résultat fondamental en analyse réelle qui stipule que toute suite réelle bornée admet au moins une sous-suite convergente. Par conséquent, toute suite réelle bornée admet au moins une valeur d'adhérence.

L'objectif de ce TP est de développer un algorithme en Python pur, "from scratch", pour identifier de manière *empirique* les valeurs d'adhérence potentielles d'une suite finie. Cette approche ne sera pas une preuve formelle, mais une méthode d'exploration numérique basée sur la détection de "clusters" de points.

Code Python

Nous allons implémenter deux fonctions principales : 1. `generate_sample_sequence` : Pour créer une suite d'exemple qui possède des valeurs d'adhérence connues. 2. `find_empirical_limit_points` : L'algorithme principal pour détecter les valeurs d'adhérence.

```
from typing import List

def generate_sample_sequence(n_terms: int) -> List[float]:
    """
    Génère une suite d'exemple avec des valeurs d'adhérence connues.
    Pour ce TP, nous utilisons la suite  $u_n = (-1)^n + 1/n$  pour  $n \geq 1$ .
    Cette suite est bornée et admet deux valeurs d'adhérence : -1 et 1.

    Args:
        n_terms: Le nombre de termes à générer pour la suite.

    Returns:
        Une liste de flottants représentant les termes de la suite.
    """
    # Validation des entrées
    if not isinstance(n_terms, int) or n_terms <= 0:
        raise ValueError("n_terms doit être un entier positif.")

    sequence: List[float] = []
    for n in range(1, n_terms + 1):
        term = float((-1)**n + (1 / n))
        sequence.append(term)
    return sequence

def find_empirical_limit_points(
    sequence: List[float],
    epsilon: float,
    min_cluster_size: int
) -> List[float]:
    """
    Recherche empiriquement les valeurs d'adhérence d'une suite donnée.
    Une valeur est considérée comme une valeur d'adhérence potentielle
    si un nombre suffisant de termes de la suite se trouvent dans son
    voisinage (défini par epsilon).

    Args:
        sequence: La liste des termes de la suite.
        epsilon: Le rayon du voisinage à considérer autour de chaque point
            pour former un cluster. Doit être > 0.
        min_cluster_size: Le nombre minimum de points requis dans un
    """
```

```

        voisinage pour qu'il soit considéré comme un cluster.
        Doit être > 0.

Returns:
    Une liste de flottants représentant les valeurs d'adhérence empiriques
    trouvées. Les valeurs sont arrondies pour éviter les duplications
    numériques proches.
"""
# Validation des entrées
if not isinstance(sequence, list) or not sequence:
    raise ValueError("La suite ne doit pas être vide.")
if not all(isinstance(x, (int, float)) for x in sequence):
    raise TypeError("Tous les éléments de la suite doivent être numériques.")
if not isinstance(epsilon, (int, float)) or epsilon <= 0:
    raise ValueError("epsilon doit être un nombre strictement positif.")
if not isinstance(min_cluster_size, int) or min_cluster_size <= 0:
    raise ValueError("min_cluster_size doit être un entier strictement positif.")

empirical_limit_points: List[float] = []
# Utilisation d'un ensemble pour stocker les points déjà "couverts"
# afin d'éviter de rapporter des points très proches comme distincts.
# On stocke une version arrondie pour la comparaison.
covered_points_rounded: List[float] = []
rounding_precision = -int(epsilon_to_precision(epsilon)) # Détermine la précision
d'arrondi

for x in sequence:
    cluster_count = 0
    for y in sequence:
        if abs(x - y) < epsilon:
            cluster_count += 1

    if cluster_count >= min_cluster_size:
        # Ce point 'x' est au centre d'un cluster.
        # Vérifions s'il est déjà représenté dans nos valeurs d'adhérence trouvées.
        is_already_covered = False
        rounded_x = round(x, rounding_precision) # Arrondir x pour la comparaison

        for existing_rounded_point in covered_points_rounded:
            if abs(rounded_x - existing_rounded_point) < epsilon: # Comparaison
                avec epsilon
                is_already_covered = True
                break

        if not is_already_covered:
            empirical_limit_points.append(x)
            covered_points_rounded.append(rounded_x)

# Pour une meilleure présentation, trier et arrondir les résultats finaux.
# On arrondit à la précision déterminée par epsilon.
final_points = sorted([round(p, rounding_precision) for p in
    empirical_limit_points])

# Éliminer les doublons après arrondi si epsilon est grand
unique_final_points: List[float] = []
if final_points:
    unique_final_points.append(final_points[0])
    for i in range(1, len(final_points)):
        if abs(final_points[i] - unique_final_points[-1]) >= epsilon:
            unique_final_points.append(final_points[i])

    return unique_final_points

def epsilon_to_precision(epsilon: float) -> int:
    """

```

```

Convertit un epsilon en un nombre de décimales pour l'arrondi.
Par exemple, si epsilon est 0.1, la précision est 1 décimale.
Si epsilon est 0.001, la précision est 3 décimales.
"""
if epsilon <= 0:
    return 0
precision = 0
temp_epsilon = epsilon
while temp_epsilon < 1:
    temp_epsilon *= 10
    precision += 1
return precision

# --- Exemple d'utilisation ---
if __name__ == "__main__":
    print("--- TP 5: Recherche Empirique des Valeurs d'Adhérence ---")

    # 1. Génération de la suite
    num_terms = 1000
    try:
        sequence_example = generate_sample_sequence(num_terms)
        print(f"\nSuite générée (premiers 10 termes): {sequence_example[:10]}...")
        print(f"Suite générée (derniers 10 termes): ...{sequence_example[-10:]}")
    except ValueError as e:
        print(f"Erreur lors de la génération de la suite: {e}")
        exit()

    # 2. Recherche des valeurs d'adhérence
    # Paramètres empiriques
    epsilon_val = 0.05 # Rayon du voisinage
    min_cluster = num_terms // 50 # Au moins 2% des termes doivent être dans le
    voisinage

    print(f"\nParamètres de recherche: epsilon={epsilon_val}, min_cluster_size={
        min_cluster}")

    try:
        found_limit_points = find_empirical_limit_points(
            sequence_example,
            epsilon_val,
            min_cluster
        )
        print(f"\nValeurs d'adhérence empiriques trouvées: {found_limit_points}")
        print(f"\nValeurs d'adhérence théoriques attendues pour  $u_n = (-1)^n + 1/n$  :
            [-1.0, 1.0]")

    except (ValueError, TypeError) as e:
        print(f"Erreur lors de la recherche des valeurs d'adhérence: {e}")

    # Test avec des paramètres différents
    print("\n--- Test avec des paramètres différents ---")
    epsilon_val_2 = 0.01
    min_cluster_2 = num_terms // 100
    print(f"Paramètres de recherche: epsilon={epsilon_val_2}, min_cluster_size={
        min_cluster_2}")
    try:
        found_limit_points_2 = find_empirical_limit_points(
            sequence_example,
            epsilon_val_2,
            min_cluster_2
        )
        print(f"Valeurs d'adhérence empiriques trouvées: {found_limit_points_2}")
    except (ValueError, TypeError) as e:
        print(f"Erreur lors de la recherche des valeurs d'adhérence: {e}")

    # Test avec une suite vide (doit lever une erreur)

```

```

print("\n--- Test d'erreur: suite vide ---")
try:
    find_empirical_limit_points([], 0.1, 10)
except ValueError as e:
    print(f"Erreur attendue: {e}")

# Test avec epsilon <= 0 (doit lever une erreur)
print("\n--- Test d'erreur: epsilon non positif ---")
try:
    find_empirical_limit_points([1.0, 2.0], 0.0, 10)
except ValueError as e:
    print(f"Erreur attendue: {e}")

```

Explications

1. Le Concept de Valeur d'Adhérence

Une valeur d'adhérence L d'une suite (u_n) est un point vers lequel une infinité de termes de la suite se "rapprochent". Cela ne signifie pas que la suite elle-même converge vers L , mais qu'une *sous-suite* de (u_n) converge vers L . Par exemple, la suite $u_n = (-1)^n$ a deux valeurs d'adhérence : -1 (pour la sous-suite des termes d'indice impair) et 1 (pour la sous-suite des termes d'indice pair).

2. L'Approche Empirique

Puisque nous travaillons avec une suite finie (bien que potentiellement très longue), nous ne pouvons pas prouver formellement l'existence d'une infinité de termes dans un voisinage. Notre approche est donc *empirique* : nous cherchons des "clusters" (regroupements) de points. Si un point x de la suite a un grand nombre d'autres points de la suite dans son voisinage immédiat (défini par `epsilon`), alors x est un bon candidat pour être proche d'une valeur d'adhérence.

3. Détails de l'Algorithme `find_empirical_limit_points`

1. **Validation des Entrées** : La fonction commence par des assertions strictes pour garantir que les paramètres `sequence`, `epsilon` et `min_cluster_size` sont valides. C'est une bonne pratique en développement logiciel pour assurer la robustesse du code.

2. **Itération sur la Suite** : L'algorithme parcourt chaque terme x de la `sequence`. Pour chaque x , il va tenter de déterminer s'il fait partie d'un cluster significatif.

3. **Détection de Cluster** : Pour chaque x , un second parcours de la `sequence` est effectué. On compte combien de termes y sont à une distance inférieure à `epsilon` de x (c'est-à-dire `abs(x - y) < epsilon`). Ce compteur est `cluster_count`.

4. **Critère de Cluster** : Si `cluster_count` atteint ou dépasse `min_cluster_size`, alors x est considéré comme le centre d'un cluster. Cela suggère que x est proche d'une valeur d'adhérence.

5. **Élimination des Doublons et Consolidation** : * Un défi majeur est que plusieurs points x et x' très proches l'un de l'autre pourraient chacun être identifiés comme centre d'un cluster, alors qu'ils représentent la même valeur d'adhérence. * Pour gérer cela, nous utilisons une liste `covered_points_rounded`. Avant d'ajouter un nouveau point x à `empirical_limit_points`, nous vérifions si une version arrondie de x est déjà "couverte" par un point existant dans `covered_points_rounded` (c'est-à-dire si `abs(rounded_x - existing_rounded_point) < epsilon`). * La fonction `epsilon_to_precision` aide à déterminer une précision d'arrondi pertinente basée sur `epsilon`. Si `epsilon` est 0.1, arrondir à 1 décimale est logique. Si `epsilon` est 0.001, arrondir à 3 décimales est approprié. * Enfin, les points trouvés sont triés et une dernière passe est effectuée pour s'assurer qu'aucun point final n'est trop proche d'un autre, consolidant ainsi les résultats.

4. La Suite d'Exemple `generate_sample_sequence`

La suite $u_n = (-1)^n + \frac{1}{n}$ pour $n \geq 1$ est un excellent exemple pour ce TP. * Lorsque n est pair, $u_n = 1 + \frac{1}{n}$. À mesure que $n \rightarrow \infty$, $u_n \rightarrow 1$. * Lorsque n est impair, $u_n = -1 + \frac{1}{n}$. À mesure que $n \rightarrow \infty$, $u_n \rightarrow -1$. Cette suite est bornée (tous ses termes sont dans $[-2, 2]$ pour $n \geq 1$) et, conformément au théorème de Bolzano-Weierstrass, elle possède des valeurs d'adhérence, qui sont précisément -1 et 1 .

5. Limitations de l'Approche Empirique

* **Dépendance à epsilon** : Si `epsilon` est trop grand, l'algorithme pourrait fusionner des valeurs d'adhérence distinctes qui sont proches. S'il est trop petit, il pourrait ne pas détecter de clusters ou en détecter trop peu. * **Dépendance à `min_cluster_size`** : Ce paramètre est crucial. S'il est trop faible, des fluctuations aléatoires pourraient être interprétées comme des valeurs d'adhérence. S'il est trop élevé, de véritables valeurs d'adhérence pourraient être manquées. ***Nature Finie de la Suite** : Une suite finie ne peut pas avoir une "in-finité" de termes dans un voisinage. L'algorithme simule cette idée en exigeant un nombre suffisant* de termes.* * **Pas une Preuve Formelle** : Cette méthode est un outil d'exploration numérique, pas une démonstration mathématique rigoureuse de l'existence de valeurs d'adhérence.

Ce TP illustre comment des concepts d'analyse mathématique peuvent être traduits en algorithmes concrets, même avec les contraintes du "from scratch", et met en lumière les compromis inhérents aux approches numériques.